



Azure Integration Framework

Technical Design Document



Document control

Change record

Version	Author	Date	Revision
0.1	Kevin Agius		Initial Draft
0.2	Benjamin Axiaq	06 Jan 2023	Added draft for Resource Groups, Storage, Logic Apps, Functions
0.3	Benjamin Axiaq	16 Jan 2023	Added table changes needed for INT34 and INT01 changes. Removed obsolete configaiIntegrationFileType table.
0.4	Jesmond Darmanin	17 Jan 2023	Added deployment information for Azure infrastructure
0.5	Benjamin Axiaq	19 Jan 2023	Added Integration Process Flows and Monitoring
0.6	Kevin Agius	03 Feb 2023	Added the Azure DevOps Technical overview
0.7	Kevin Agius	07 Feb 2023	Added the Integration Framework High Level Overview; Procurement API Process flow section; SFTP Upload Process flow; SFTP Download Process flow.
0.8	Kevin Agius	8 Feb 2023	Added the Integration Solution section
0.9	Benjamin Axiaq	8 Feb 2023	Added FAQ answer. Added statuses of status tracker table.
0.10	Benjamin Axiaq	9 Feb 2023	Updated outbound workflow to include sending Downloaded status.
0.11	Benjamin Axiaq	14 Feb 2023	Added more FAQs
0.12	Benjamin Axiaq	16 Feb 2023	Added Key vault secret list.
0.13	Benjamin Axiaq	21 Feb 2023	Added Logic App Parameters
0.14	Benjamin Axiaq	23 Feb 2023	Added FAQ about authorizing email.

0.15	Kevin Agius	21 Mar 2023	Added the Azure Function and Logic App Release processes.
0.16	Jesmond Darmanin	28 Mar 2023	Added the Infrastructure and Networking sections to the document.
0.17	Kevin Agius	28 Mar 2023	Updated the Key Vault and Logic App parameter sections.
0.18	Benjamin Axiaq	29 March 2023	Updated table fields with new additions. Updated D365FO screenshots.
0.19	Kevin Agius	29 March 2023	Added new section 12.1 Application Insights.
0.20	Jesmond Darmanin	20 April 2023	Added information about the SSL Certificate
0.21	Kevin Agius	25 April 2023	Added new section about the Application Registrations.
0.22	Kevin Agius	26 May 2023	Added new section '14.6 How to restart a Logic App?'

Reviewers

This document has been reviewed by:

	Reviewer	Last reviewed date
1	EeJie Tan	
2	Joop Siem	

Approvals

This document will be approved by:

	Name	Signature	Date approved
1			

Scope of this document

The scope of this document is to support the Azure Integration implementation of both the Collosseum Dental Group D365FO and to the Collosseum Dental Group (CDG) third parties. This document contains a high-level explanation of every Azure resource deployed and configured on the Azure Collosseum Dental Group subscriptions.

This document also contains information regarding the Azure DevOps code repositories and a high-level information regarding the Azure DevOps Build and Release pipelines.

Additionally, this document gives a clear overview of the Azure resources implemented to support the Azure Integration Solution for CDG. It contains information on the Azure Storage Account Tables, Azure Storage Account Blobs, Azure Storage Account Queues, Azure Functions, Azure Logic Apps, Azure Key Vault, Monitoring, Security, Troubleshooting methods and User Guide.

Table of Contents

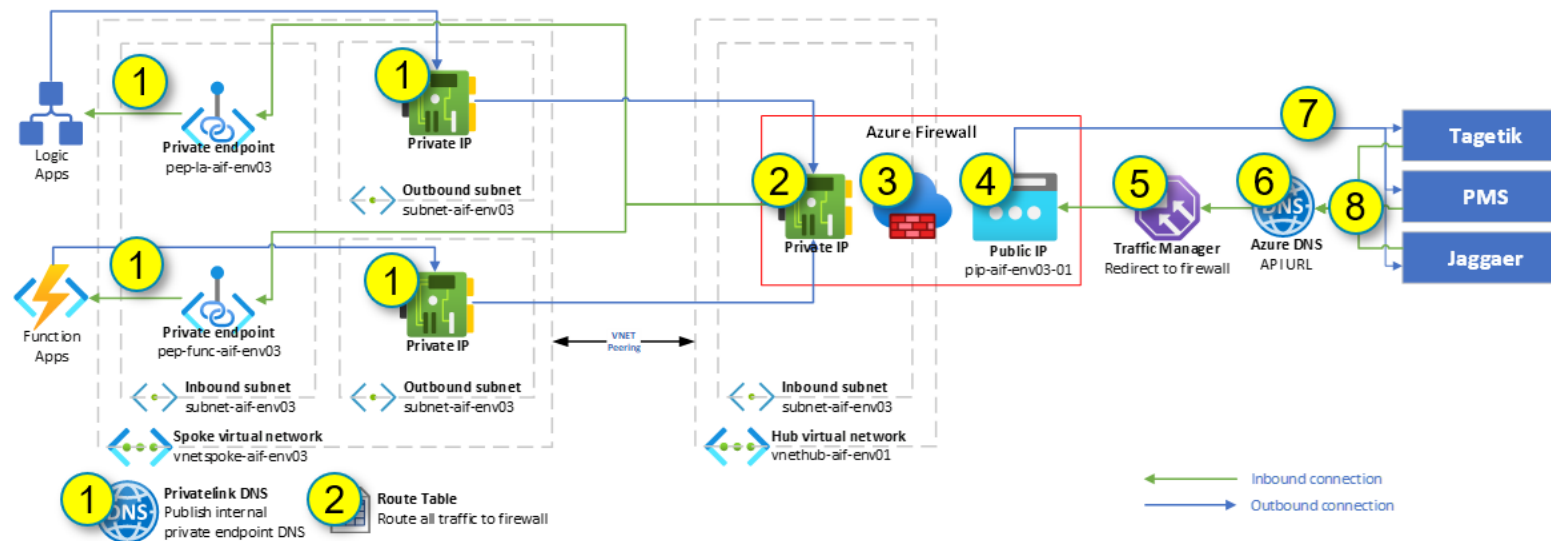
Change record		1
Reviewers		3
Approvals		3
1	Integration Solution	1
1.1	High Level Architecture	1
1.2	System Dependency	3
1.3	System Design	6
2	Azure DevOps Technical Overview	9
2.1	Azure Integration source control repository	9
2.2	Azure Integration Pipelines	13
3	Integration Process Flow	29
4	Azure Resource Groups	33
5	Azure Storage Account	34
5.1	Queues	34
5.2	Blob Containers	35
5.3	Tables	36
6	Azure Logic App Standard	49
6.1	Inbound Integration Workflows	49
6.2	Outbound Integration Workflows	51
6.3	Shared Workflows	52
6.4	Logic App Parameters	52
7	Azure Functions	55
8	File Download Process flow	56

9	File Upload Process flow	57
10	Procurement API Process flow	58
11	D365FO status monitoring	59
11.1	Inbound Integration Status Monitoring	59
11.2	Outbound Integration Status Monitoring	59
12	Monitoring	61
12.1	Status Tracker Table	61
12.2	Azure Function Logs	63
12.3	Application Insights	65
13	Security	71
13.1	VNET	71
13.2	Azure Key Vault	71
13.3	Application Registrations	75
13.4	SSL Certificate	76
14	FAQs	78
14.1	How to change the Third Party credentials	78
14.2	How to run an integration manually	80
14.3	How to monitor an integration.	81
14.4	Emails are not being sent anymore. What should I do?	83
14.5	How to Approve an Azure DevOps Release	84

1 Integration Solution

1.1 High Level Architecture

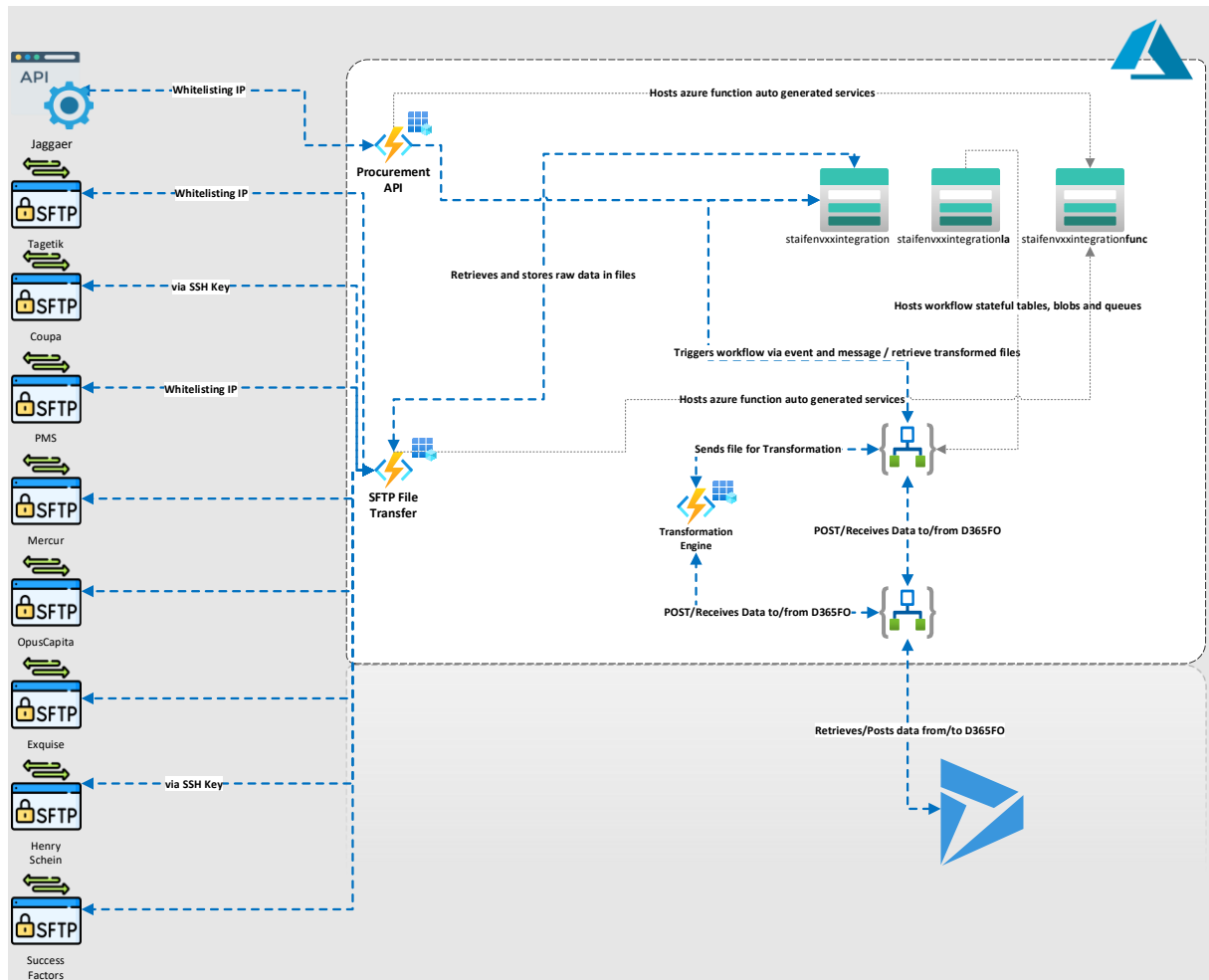
1.1.1 Network Topology



- 1 Isolate services from public internet by adding private endpoints in specific subnet in VNET and registered in private DNS Zone
- 2 Create Route table rule to direct all traffic via Firewall
- 3 Utilize Firewall for whitelisting incoming connections and providing single public IP for whitelisting by clients
- 4 Single Public IP will be used for all incoming and outgoing connections

- 5 Traffic Manager to publish URLs for APIs
- 6 Publish public alias DNS records to direct traffic to Traffic Manager endpoints (Option)
- 7 All outgoing connections
- 8 All incoming connections via Public DNS record to Firewall

1.1.2 Integration Framework High Level Overview



1.2 System Dependency

In the section below, you will find all the dependencies that the middleware solution will be tightly coupled with, hence this means that if there is a change in the below dependencies, it can lead for the middleware solution to fail to process.

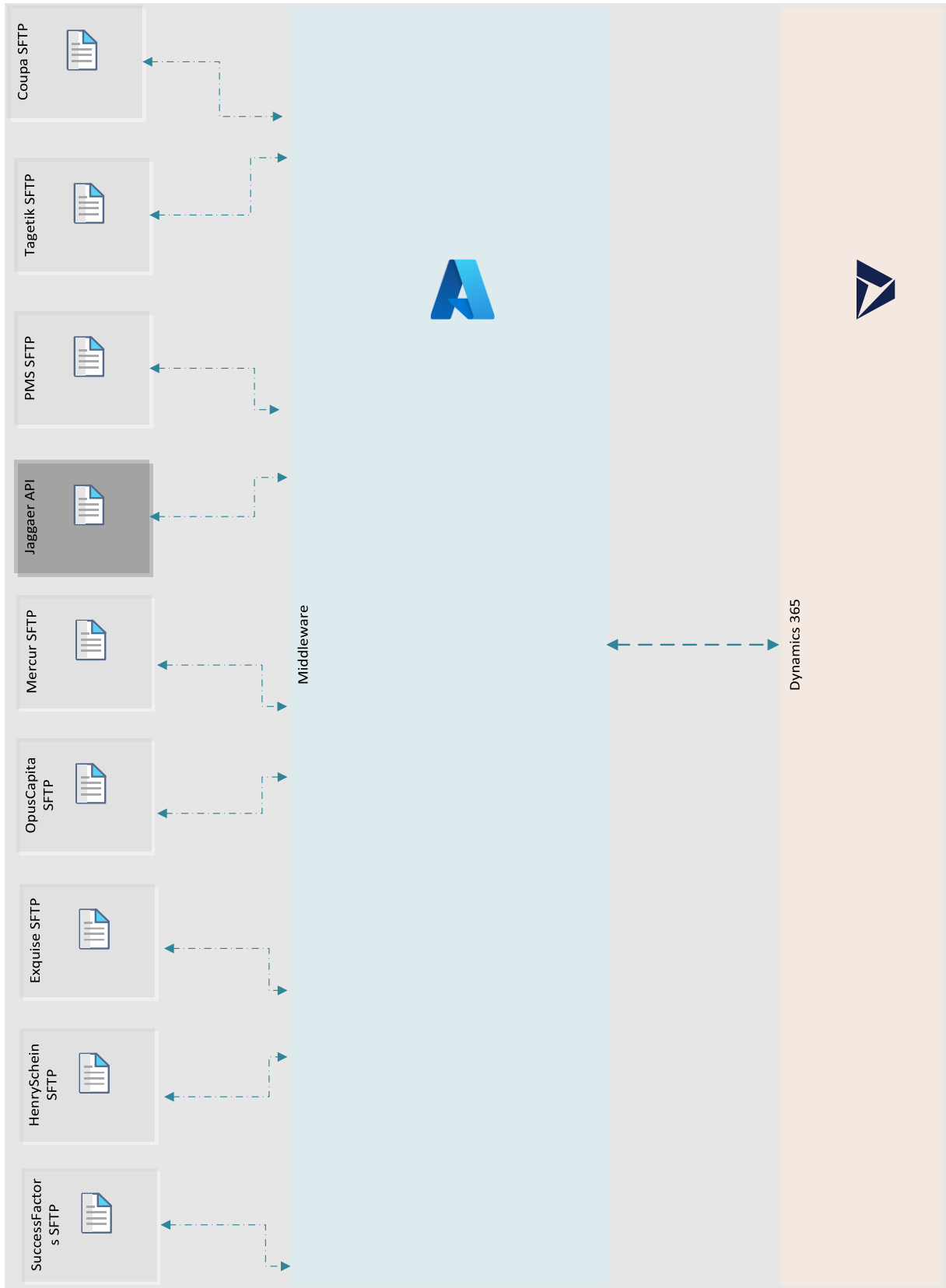
1.2.1 Endpoints dependency

The middleware solution will be connected and authenticated with several endpoints both third-party endpoints such as SFTPs and client endpoints such as the D365FO environment.

There are plenty of communication mechanisms, and whenever there is an integration, the cloud middleware needs to relate to a third-party endpoint and is not expected to change. If the endpoint changes its API or its connection mechanism, the integration will break and hence it will need to be manually changed. This also applies when the third party SFTPs changes their SFTP connection credentials.

? How to change the Third Party endpoint configurations?

Refer to section [How to change the Third Party credentials?](#)



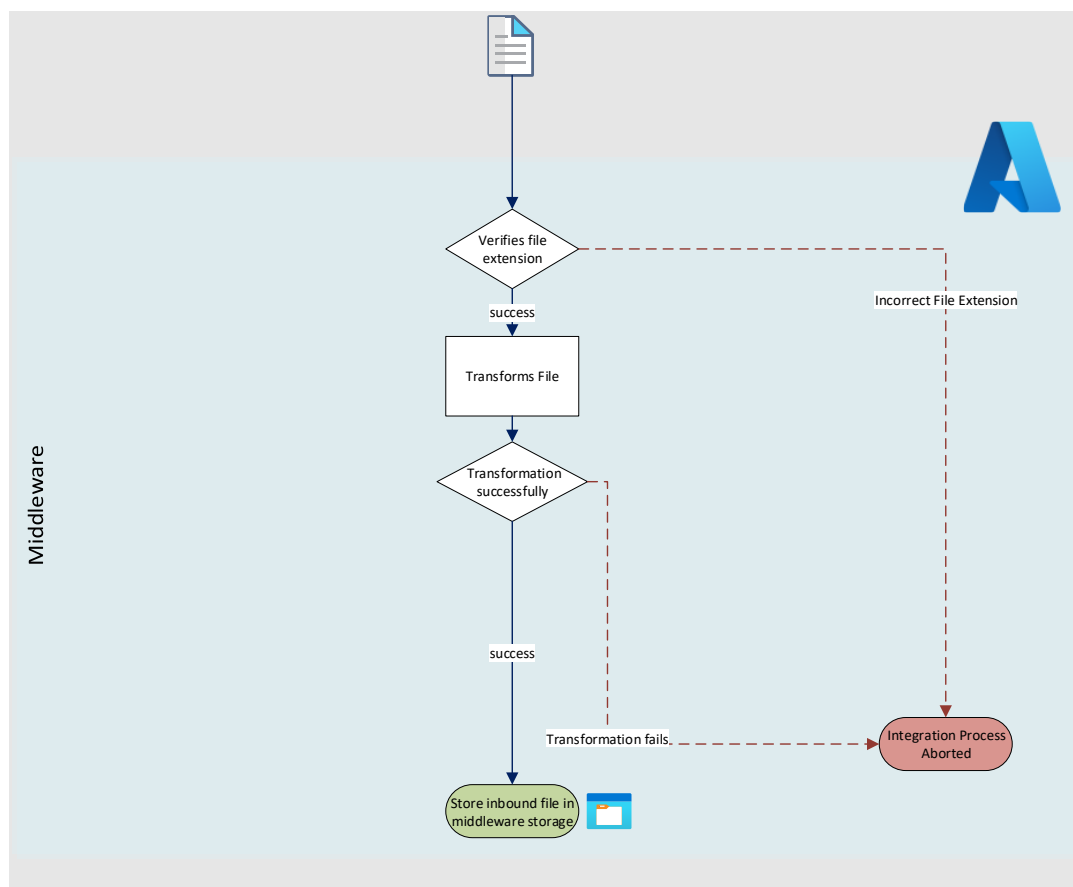
1.2.2 Data Format dependency

Each integration consists of either an API call or a native source file. Each integration artefact (i.e., file) needs to have its file format consistent across each country in every integration.

The middleware is only responsible to validate whether the file extension corresponds with the native format of the file. *Example if the client uploads a json file, the middleware will expect to find json syntax in the file, if it doesn't it will throw an error.* The transformation will log an error message and the logic app will be terminated with an error. Once the logic terminates with an error, an alert will be sent. Once this happens, the client would need to re-upload the integration source file in its source location, and it will be fetched by the middleware in the subsequent runs to re-trigger the integration.

If the file format corresponds with the extension, it will transform the file from the inbound source file to D365FO format.

No additional validations are needed in the Middleware, hence any issues with the file format/incorrect parameters will be handled by the Data Management Framework in D365FO.



1.3 System Design

1.3.1 Naming Conventions

This section contains a list of naming conventions of all the different type of resources that are going to be deployed and hosted in the Middleware solution (i.e., Azure Subscription).

The **Production** and **Non-Production** resources are hosted on two separate CDG Azure subscriptions as per requested by the client:

Common ERP Beta	1be4c9c7-942b-4420-99d2-f183e515d961
Common ERP Prod PAYG	8937fe28-3278-4a2a-af9f-bd40925f924f

Each Azure middleware is going to be hosted underneath a specific Azure resource group and each resource group is going to follow the below naming convention standards to define the purpose and environment from one resource group to another.

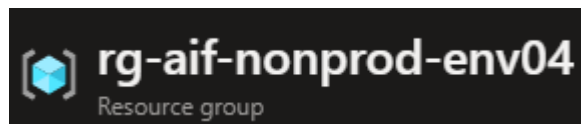
1.3.1.1 Middleware abbreviation

The below middleware abbreviation is going to be used to define the purpose of each resource and for the client to distinguish the middleware resource from other resources that are hosted in the Azure subscription.

The term '**aif**' stands for *Azure Integration Framework*.

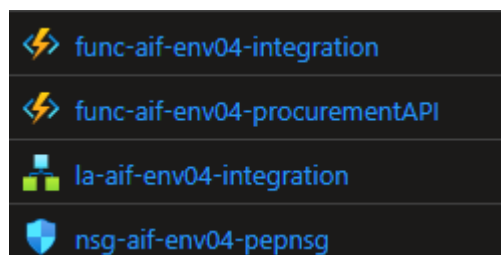
Resource Group:

<resource abbreviation>--**aif**--<prod/nonprod>--<env abbreviation>



Resources inside the resource group:

<resource abbreviation>--**aif**--<env abbreviation>--<scope>



1.3.1.2 Environment abbreviations

This section contains a list of environments with their abbreviations. The below list covers all the integrated environments that are currently deployed and used in this middleware implementation.

Middleware environment reference	Resource group reference
03	rg-aif-nonprod-env03
04	rg-aif-nonprod-env04
05	rg-aif-nonprod-env05
06	rg-aif-nonprod-env06
01	rg-aif-prod-env01

1.3.1.3 Resources naming convention

The below are the naming conventions used to deploy resources on the CDG subscriptions.

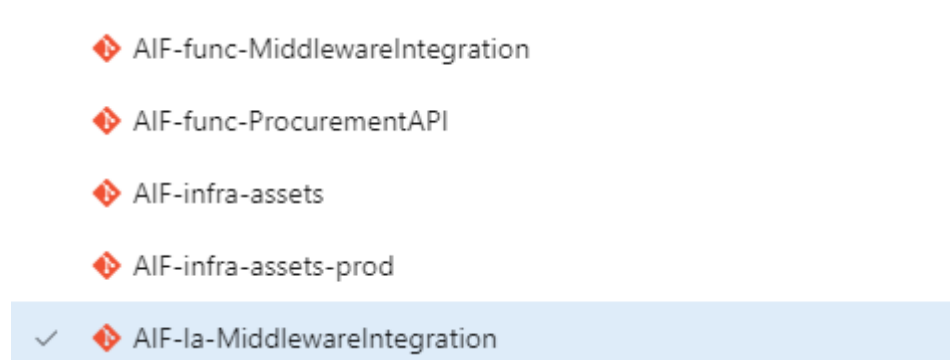
Type	Name	Comment
App Service Plan	plan-aif-<env>-<purpose>	<env> defines the environment reference. (e.g., env03) <purpose> defines the type of App Service Plan. (e.g., plan-aif-env03-functions or plan-aif-env03-logicapps)
Logic App	la-aif-<env>-<purpose>	<env> defines the environment reference. (e.g., env03) <purpose> defines the name of the logic app.
Azure Function	func-aif-<env>-<purpose>	<env> defines the environment reference. (e.g., env03) <purpose> defines the name of the Azure Function. (e.g., func-aif-env03-integration)
Storage Account	staif<env><purpose>	<env> defines the environment reference. (e.g., env03) <purpose> defines the name of the storage account. (e.g., staifenv03integration)
Key Vault	kv-aif-<prod nonprod>	<prod nonprod> - defines whether the environment is nonproduction or production environment.
Application Insights	appi-aif-<prod nonprod>-<purpose>	<prod nonprod> - defines whether the environment is nonproduction or production environment. <purpose> defines the type of Application Insights (e.g., appi-aif-env03-functions or appi-aif-env03-logicapps)
Virtual Network	vnet-aif-<env>	<env> defines the environment reference. (e.g., env03)
Private endpoint	pl-aif-<env>-<resourcetype>-<type>-endpoint	<env> defines the environment reference. (e.g., env03) <resourcetype> defines whether it's a private or public endpoint. <type> defines the type of endpoint (e.g., private, or public)

2 Azure DevOps Technical Overview

Colosseum Dental Group (CDG) is using Azure DevOps to host AIF Middleware source control, version control, automated builds, and release management. In this DevOps Tenant you will find several source code repositories using *git*.

2.1 Azure Integration source control repository

CDG Azure solution contains five Azure DevOps *git* code repositories. These are as follows:



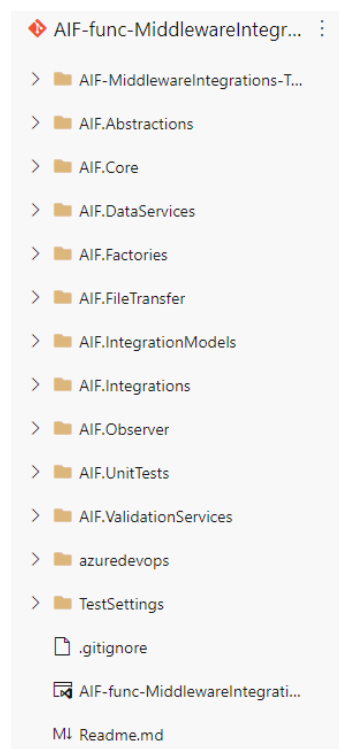
2.1.1 AIF-func-MiddlewareIntegration

The integration transformation library its hosted underneath this *git* source control repository.

This library is composed by a number of .NET Core projects which are being used to transform the files for all the integrations that requires a file transformation. Additionally this library is also being used to Authenticate, Upload and Download files via SFTP. The .NET Core library is being invoked via an Azure Function (i.e. func-aif-envxx-integration).

In this repository we are also hosting the Azure DevOps Build YAML pipeline which can be found underneath the *azureddevops* folder.

Refer to [section 5](#) for information on the Azure Function.



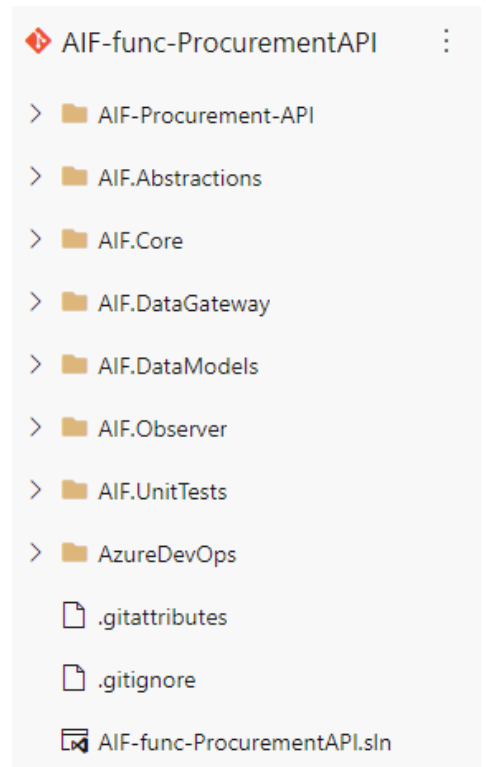
2.1.2 AIF-func-ProcurementAPI

This source control repository stores the library and the API which were designed and developed to be used by third parties to send procurement related data via an API.

The library is composed by a number of .NET Core projects which are being used to authenticate and send API calls from the third party to the AIF Middleware.

This API was originally designed and developed specifically for Jaggaer (i.e. *A Third Party that several countries within Collosseum Dental Group were/are using to handle procurements*).

In this repository we are also hosting the Azure DevOps Build YAML pipeline which can be found underneath the *azuredevops* folder.



Refer to [section 5](#) for information on the Azure Function.

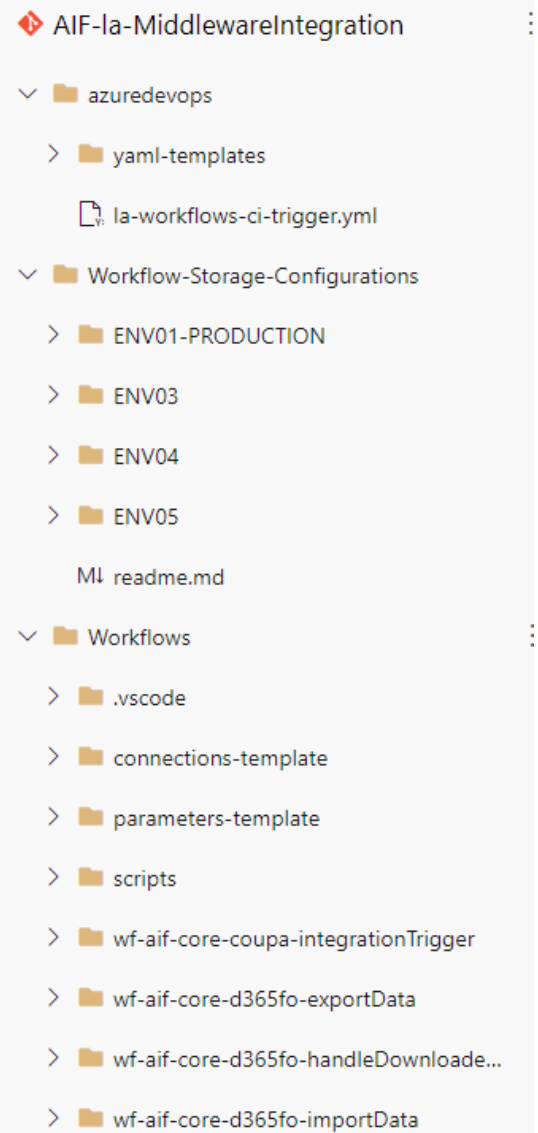
2.1.3 AIF-Ia-MiddlewareIntegration

This source control repository stores the logic app workflow code; storage account table configurations and the CI/CD pipelines used to deploy the logic app standard workflows.

Underneath the *azuredevops* directory you will find the CI and CD YAML pipelines used to build and release the logic app workflows to a number of CDG environments.

Underneath the *Workflow-Storage-Configurations* directory; you will find a folder per each environment. Underneath each folder you will find a .csv file per each azure storage account table. The scope of this directory is to be used by the BAU / Azure engineer to be able to restore the configurations in case of any Disaster Recovery Failover plan.

Underneath the *Workflows* directory; you will find a folder per each workflow that contains the *workflow.json* file. This file contains the logic app code. In this directory, you will also find the *connections-template*, *parameters-template*, *scripts*, and other folders which we will write more in detail in the subsequent chapters. The latter folders are used for the configuration transformation (environment specific transformations) operations which are done during the build process and the deployment of the logic app workflow solution on several CDG environments.



2.1.4 AIF-infra-assets & AIF-infra-assets-prod

AIF-infra-assets

- Master branch is ENV04
- UAT branch is ENV05 (UAT) branch
- Main branch is not used

AIF-infra-assets-prod

- Master branch is the production environment as well as shared resources
- Main branch is not used

Note: Network resources are shared, and they are deployed from the production environment only.

The repository is a mix of Terraform (.tf) files and Azure Arm Templates (.json) files. Some resources could not be created using Terraform when the scripts were developed. Therefore, they had to be built using Arm Templates.

If an ARM template is modified, the terraform state will not change and therefore the modification will not be applied. To force the creation of the resource, delete the resource itself from Azure.

Once deleted, go to the resource group and find the deployment file of the resource and delete it too.

Terraform files of interest are:

- Backend.tf: This needs to be changed when creating a new environment.
- Secrets*.tf: This file contains the key vault secrets. Please note that terraform script do not create all secrets for the project. Some secrets used by D365 must be created manually.
- Apps.tf & Appsnoprod.tf: Most of the azure resources are in these scripts.

2.2 Azure Integration Pipelines

2.2.1 Environments

Environments are being used to store information of all the DevOps releases deployed on each middleware environment.

Till to date, CDG have four environments:

Environment name	Purpose
ENV04-INT	Contains a list of releases deployed to the ENV04 middleware (i.e., Internal testing environment).
ENV05-UAT	Contains a list of releases deployed to the ENV05 middleware (i.e.: User Acceptance environment).
ENV06-PREPROD	Contains a list of releases deployed to the ENV06 middleware (i.e. Pre-Production environment).
ENV01-PRODUCTION	Contains a list of releases deployed to the ENV01 middleware (i.e., Production environment).

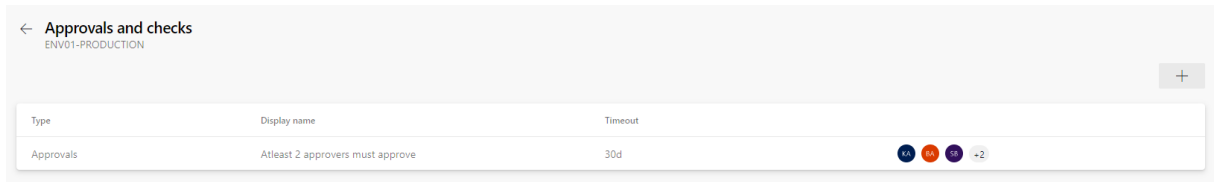
To access the Azure DevOps Environments:

- Click on Pipelines
- Click on Environments.

Environments	
Environment	Status
ENV01-PRODUCTION	✓ #20230410.1 on AIF-Ia-MiddlewareIntegration-Release
ENV04-INT	✓ #20230419.1 on AIF-Ia-MiddlewareIntegration-Release
ENV05-UAT	✓ #20230410.1 on AIF-Ia-MiddlewareIntegration-Release
ENV06-PREPROD	Never deployed

2.2.1.1 Approvals and Checks

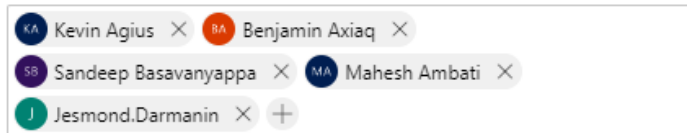
The DevOps Environments are also being used to configure the pre-deployment approvals before a logic app deployment takes place. This is similar to the pre-deployment approvals found in the Azure DevOps Releases for the Azure Functions but the process is slightly different.



Type	Display name	Timeout
Approvals	Atleast 2 approvers must approve	30d

Approvals

Approvers



Instructions to approvers (optional)

Advanced

Minimum number of approvals required

☒ Allow approvers to approve their own runs

Control options

Timeout

Fig: The above configurations were taken from the ENV01-Production Environment which is responsible to store Logic App Standard deployment information and configuration on the pre-deployment process. The pre-deployment process contains of a list of approvers, the minumum number of approvals required and the release timeout.

2.2.2 Library

A library is a set of build and release assets for an Azure DevOps project. The library assets can be accessed from both the build and release pipelines.

In this project, the Library contains a list of configurations which are required to be used during the build and release processes of both the Azure Logic App Standard and Infrastructure as Code deployments.

The configurations are stored in specific variable groups to distinguish one set of configurations from the other.

To access the Azure DevOps Library:

- Click on Pipelines
- Click on Library

Library	
Variable groups	Secure files + Variable group
Name ↓	
.fx	ENV01-Production
.fx	ENV03-Dev
.fx	ENV04-IntTst
.fx	ENV05-UAT
.fx	ENV06-PreProd
.fx	kv-aif-nonprod
.fx	kv-aif-prod
.fx	shared-infra-variables

2.2.2.1 Library Variable Groups

Below find a list of variable groups that currently there is stored in the CDG Library:

Variable Group	Purpose
ENV03-DEV	Contains the Logic App standard ENV03 configurations (i.e., Development Logic App environment).
ENV04-IntTst	Contains the Logic App standard ENV04 configurations (i.e., Internal Testing Logic App environment).
ENV05-UAT	Contains the Logic App standard ENV05 configurations (i.e., User Acceptance Testing Logic App environment).
ENV06-PreProd	Contains the Logic App standard ENV06 configurations (i.e., Pre-Production Logic App environment).
ENV01-Production	Contains the Logic App standard ENV01 configurations (i.e., Production Logic App environment).
kv-aif-nonprod	Contains the Infrastructure as Code configurations for all the non-production resources.
kv-aif-prod	Contains the Infrastructure as Code configurations for all the production resources.
shared-infra-variables	This contains common variables that are used across the various environments. It contains mainly networking information for the various resources. Networking is built by the production environment.

2.2.3 Pipelines

The pipelines contain a list of build and release processes developed in YAML used to (1) build the Azure Functions (i.e. *AIF-func-MiddlewareIntegration* & *AIF-func-ProcurementAPI*), (2) a build pipeline which runs through the logic app build process and (3) the release pipeline to deploy the logic app standard to all the middleware environments.

Pipelines	
Recent	All
All pipelines	
Name	
Azure	
✓	AIF-func-MiddlewareIntegration-Temp #20220905.3 • removed trigger from temp build
✓	AIF-func-MiddlewareIntegration #20230117.1 • Merged PR 485: INT01 - Upload folder depending on file name
✓	AIF-func-ProcurementAPI #20220412.1 • Merged PR 133: Add two new Feature Flags one for INT05 and for INT06. The integrations will...
✓	AIF-la-MiddlewareIntegration-Build #20230130.1 • Merged PR 496: Updated recurrence times
🕒	AIF-la-MiddlewareIntegration-Release #20230130.1 • Merged PR 496: Updated recurrence times

2.2.3.1 AIF-func-MiddlewareIntegration

This pipeline contains the steps to build and generate the build artefact of the *AIF-func-MiddlewareIntegration* resource.

The steps carried out are as follows:

- 1) Builds the *AIF-func-MiddlewareIntegration* library;
- 2) Executes a number of test cases via the Unit Test project; and
- 3) Publishes the build artifact in the staging directory.

2.2.3.2 AIF-func-ProcurementAPI

This pipeline contains the steps to build and generate the build artefact of the *AIF-func-ProcurementAPI* resource.

Note: This resource is not currently active since CDG are not currently using the Jaggaer API for procurement.

The steps carried out are as follows:

- 1) Builds the *AIF-func-ProcurementAPI* library;
- 2) Executes a number of test cases via the Unit Test project; and
- 3) Publishes the build artifact in the staging directory.

2.2.3.3 Aif-la-MiddlewareIntegration-Build

This pipeline contains the steps to build the Logic App standard (i.e. *AIF-la-MiddlewareIntegration*) and generates a build artefact for each middleware environment with specific environment configurations.

All configurations are retrieved from the [Library Variable Groups](#).

The steps carried out are as follows:

1. Generates a logic app standard *connection* file per each middleware environment;
2. Generates a logic app standard *parameter* file per each middleware environment;
3. Creates the deployable artefact per each middleware environment;
4. Compresses the deployable artefact with the name of Workflows_\$(Build.BuildNumber).zip';
5. Publishes a deployable artefact (package) per each middleware environment.

Note: The scope of this pipeline is to run a set of configuration transformations to generate the artefact to be deployed. The deployment of the logic apps is done through another pipeline.

2.2.3.4 Aif-la-MiddlewareIntegration-Release

The scope of this pipeline is to deploy the deployable artefact generated by the [Aif-la-MiddlewareIntegration-Build](#) pipeline to a number of middleware environments.

This pipeline contains three stages and in each stage the pipeline is deploying the specific deployable artefact (package) to the specific logic app standard resource. Each release is deployed against the specific [DevOps configured environment](#). Hence there might be stages wherein it might require a pre-approval prior the deployment takes place.

Stage Identifier	Purpose
INTTEST_Deployment	The stage that contains the job to deploy to the Internal Testing environment (i.e. ENV04) using the self-hosted agent.
UAT_Deployment	The stage that contains the job to deploy to the User Acceptance environment (i.e. ENV05) using the self-hosted agent.
PREPROD_Deployment	The stage that contains the job to deploy to the Pre-Production environment (i.e., ENV06) using the self-hosted agent.
PRODUCTION_Deployment	The stage that contains the job to deploy to the Production environment (i.e., ENV01) using the self-hosted agent.

2.2.3.5 AIF-NonProd-Terraform-Plan

This pipeline contains the steps to build the Env04 terraform artifact. This pipeline will reference to the code in the repository *AIF-Infra-Assets* Master branch. Once code is committed to the repository, the pipeline triggers the build automatically.

The steps carried out are as follows:

1. Install Terraform. This will install terraform to the current runtime instance.
2. Terraform Init. Initialises the terraform with the credentials stored in the key vault secret connected to the variable group kv-aif-nonprod
3. Terraform validate. Validates the configuration files of terraform
4. Terraform Plan. This will generate the build plan. The plan command accepts inputs from variables stored in the variable groups kv-aif-prod and shared-infra-variables. The terraform plan will contain any errors related to syntax of the terraform script or state. The plan will also show what resources will be added, deleted or changed. Always check the plan before proceeding to release.
5. Archive Terraform Plan. This will prepare a zip file out of the plan.
6. Publish Terraform Plan. Save the plan zip file for use by the release pipelines.

2.2.3.6 AIF-NonProd-Terraform-Plan-Env05

This pipeline will deploy to Env05 (UAT). The steps in this pipeline are identical to AIF-NonProd-Terraform-Plan. This pipeline will reference to the code in the repository *AIF-Infra-Assets* UAT branch. The other difference is that the plan will reference env05 variables instead of env04.

2.2.3.7 AIF-Prod-Terraform-Plan

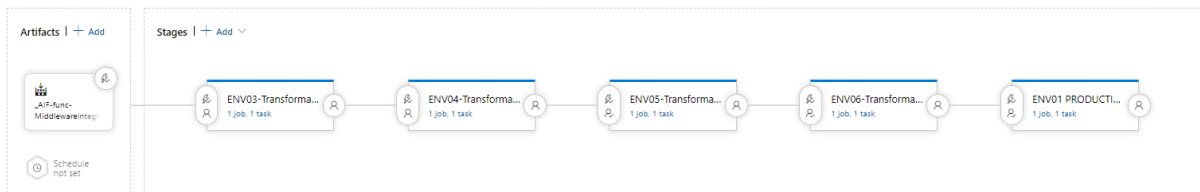
This pipeline will build the Production release. The terraform script contain the networking of the non-prod environment too. The steps in this pipeline are identical to *AIF-NonProd-Terraform-Plan*. This pipeline will reference to the code in the repository *AIF-Infra-Assets-Prod* Master branch. The other difference is that the plan will reference env01 variables instead of env04.

2.2.4 Releases

The release process contains the steps taken to deploy the build artefact on Azure. Each deployable application has a separate Release pipeline that is fully customisable through Azure DevOps > Releases.

2.2.4.1 Deploy CDG AIF Middleware Transformations Function

The scope of this release pipeline is to deploy the latest AIF-func-MiddlewareIntegration build artefact which is created throughout the [AIF-func-MiddlewareIntegration pipeline](#) to Azure.



This pipeline contains four stages that deploys sequentially to separate Azure middleware solutions.

The first stage is being deployed automatically as soon as a new build artefact is created which will deploy the artefact to ENV03 solution.

In order for the build artefact to be released to the ENV04 stage, the Azure DevOps release manager will have to manually release to this middleware solution by simply clicking the Deploy button.

The ENV05, ENV06 and ENV01 stages contains an additional step prior the artefacts are released to these environments. This additional step is known as the 'Pre-deployment approvals' wherein one or more AAD Users / AAD Groups can be configured to approve the release prior the deployment takes place.

In order to configure the Pre-deployment approvals, the release manager would need to click on the Pre-deployment conditions button and configure the Pre-deployment approvals section.

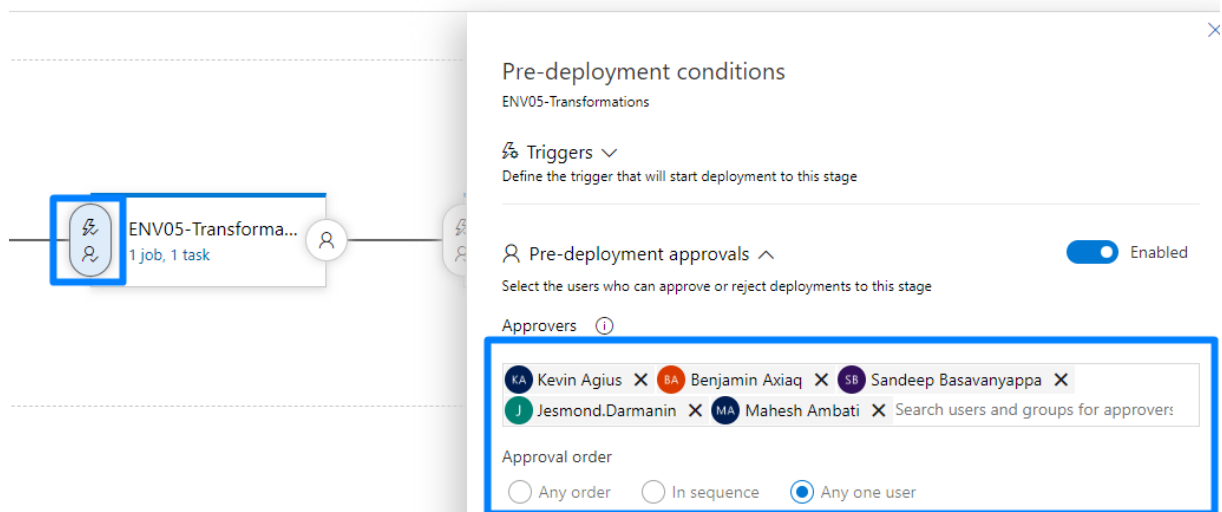


Fig: Shows the Pre-deployment approvals of ENV05 till to date. The approval order is configured to 'Any one user' hence it means that if one of the approvals listed above approves the release, it will be automatically deployed to ENV05.

The same process applies to ENV01 (Production deployment).

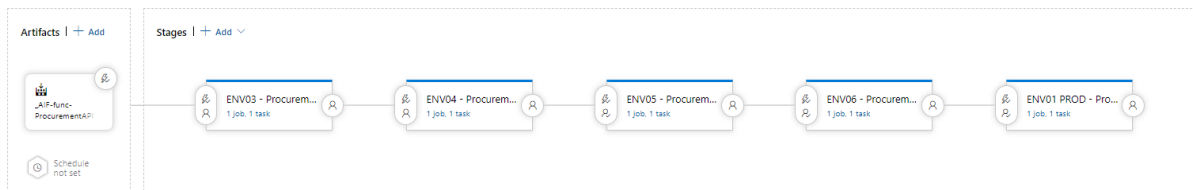
As a best practice, KPMG recommends to configure the Pre-deployment approvals using AAD Groups rather than AAD Users in order to better maintain the approvals in the long run.

? How to approve Azure DevOps Releases?

Refer to section [How to approve Azure DevOps Releases?](#)

2.2.4.2 Deploy CDF AIF Procurement API Function

The scope of this release pipeline is to deploy the latest build artefact AIF-func-ProcurementAPI which is created via the [AIF-func-ProcurementAPI pipeline](#).



This pipeline is made up of four stages which are responsible to deploy the build artefact to four separate middleware solutions. Once a new build is created via the AIF-func-ProcurementAPI pipeline, the first stage will deploy the artefact to ENV03.

In order to deploy the artefact to ENV04, the release manager would need to manually trigger the release process via this pipeline.

The ENV05, ENV06 and ENV01 stages contains an additional step prior the artefacts are released to these environments. This additional step is known as the 'Pre-deployment approvals' wherein one or more AAD Users / AAD Groups can be configured to approve the release prior the deployment takes place.

In order to configure the Pre-deployment approvals, the release manager would need to click on the Pre-deployment conditions button and configure the Pre-deployment approvals section.

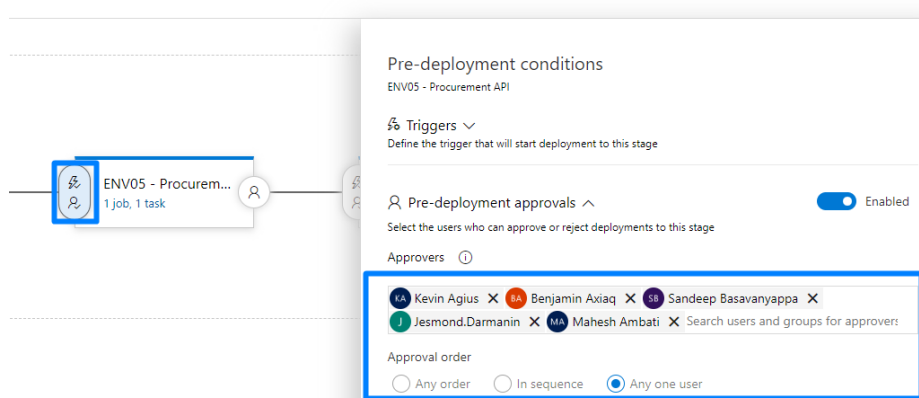


Fig: Shows the Pre-deployment approvals of ENV05 till to date. The approval order is configured to 'Any one user' hence it means that if one of the approvals listed above approves the release, it will be automatically deployed to ENV05.

The same process applies to ENV01 (Production deployment). As a best practice, KPMG recommends to configure the Pre-deployment approvals using AAD Groups rather than AAD Users in order to better maintain the approvals in the long run.

? How to approve Azure DevOps Releases?

Refer to section [How to approve Azure DevOps Releases?](#)

2.2.4.3 AIF-NonProd-Terraform-Apply

This will apply the infrastructure changes to ENV04.

The release is created as soon as there is a successful plan that produced an artifact. The release must be approved by one or more persons to be applied. Before approving a release check the changes that are going to be applied in the plan step of the build pipeline. The release is ready to be approved once there is a blue icon visible. Click on the blue icon to approve. Once it is approved, the release will be executed and report back whether it was successful or failed. Check the output to ensure that there are no errors. Also check the resource groups in the Azure Portal to ensure the changes have been applied. The resource group for this pipeline is **rg-aif-nonprod-env04**.

The screenshot displays the 'Jobs in run #2882' for the 'AIF-NonProd-Terraform-Plan' pipeline. The job list on the left shows a sequence of steps, with 'Terraform Plan' highlighted. The right pane shows the Terraform Plan output, which includes the configuration for the 'azurerm_key_vault_secret' and 'azurerm_logic_app_standard' resources. The plan summary at the bottom indicates 'Plan: 0 to add, 5 to change, 0 to destroy.'

Jobs	Duration
✓ Terraform Plan	2m 25s
✓ Initialize job	3s
⌚ Pre-job: Download se...	<1s
✓ Download secrets: kv...	<1s
✓ Checkout AIF-infra-asse...	1s
✓ Install Terraform 0.14.9	2s
✓ Terraform Init	3s
✓ Terraform Validate	1s
✓ Terraform Plan	1m 49s
✓ Archive Terraform Plan ...	7s
✓ Publish Terraform Plan ...	6s
✓ Post-job: Checkout Al...	<1s
✓ Finalize Job	<1s
✓ Report build status	<1s

```
176 ~ resource "azurerm_key_vault_secret" "logicappmasterkey" {
177     id          = "https://kv-aif-nonprod.vault.azure.net/secrets/cdg-la-aif-env04-c
178     name        = "cdg-la-aif-env04-configuration-func-functionAppKey-masterkey"
179     tags        = {}
180     ~ value      = (sensitive value)
181     # (4 unchanged attributes hidden)
182 }
183
184 # azurerm_logic_app_standard.logicapp1 will be updated in-place
185 ~ resource "azurerm_logic_app_standard" "logicapp1" {
186     ~ app_settings = {
187         + "AzureWebJobsStorage" = "@Microsoft
188         # (22 unchanged elements hidden)
189     }
190     id          = "/subscriptions/1be4c9c7-942b-4420-99d2-f183e515ds
191     name        = "la-aif-env04-integration"
192     tags        = {
193         "deployedby" = "Schalk du Plessis"
194         "environment" = "NonProd"
195         "function"   = "IaC development"
196     }
197     # (18 unchanged attributes hidden)
198
199     # (2 unchanged blocks hidden)
200 }
201
202 Plan: 0 to add, 5 to change, 0 to destroy.
203
204 -----
205
206
```

Fig: Shows a successful Terraform Plan execution. It also shows that there are 0 resources to add, 5 to change and 0 to delete.

 Pending approval on **Terraform Apply** stage.

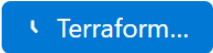
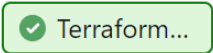
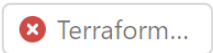
Releases	Created	Stages
 Release-47  292...  uat	28/03/2023, 11:59:17	 Terraform...
 Release-46  288...  uat	16/03/2023, 14:01:13	 Terraform...
 Release-45  284...  uat	24/02/2023, 10:00:54	 Terraform...
 Release-44  284...  uat	24/02/2023, 09:46:10	 Terraform...

Fig: The release pipeline ready to be applied (Blue button). Click on the blue button to approve. Green buttons mean that previous runs where successful. The button with red cross indicates previous errors.

Release-47 > Pending approval on Terraform Apply stage.

Pre-deployment approval pending

[View logs](#)

 Approval pending for 2 hours
Waiting for **any one** of the approvers to approve.

 Timeout in 7d



DawidSchalk.DuPlessis

 Pending for 2 hours



Jesmond.Darmanin

 Pending for 2 hours

 [Reassign](#)

Comment

☐ Defer deployment for later

Approve

Reject



Kevin Agius

 Pending for 2 hours



Benjamin Axiaq

 Pending for 2 hours

Fig: To approve the release and apply the changes click on the Approve button. Depending on the setup there could be more than one approver required.

2.2.4.4 AIF-NonProd-Terraform-Apply-ENV05

This will apply the infrastructure changes to ENV05 (UAT).

The pipeline is identical to AIF-NonProd-Terraform-Apply that update ENV04 environment. The only difference is that it is triggered by the UAT release pipeline and therefore has a different artifact. Follow the instruction as per ENV04 deployment. The resource group for this pipeline is **rg-aif-nonprod-env05**.

2.2.4.5 AIF-NonProd-Terraform-Apply-ENV06

This will apply the infrastructure changes to ENV06 (Pre-Production).

The pipeline is identical to AIF-NonProd-Terraform-Apply that update ENV04 environment. The only difference is that it is triggered by the PREPROD release pipeline and therefore has a different artifact. Follow the instruction as per ENV04 deployment. The resource group for this pipeline is **rg-aif-nonprod-env06**.

2.2.4.6 AIF-Prod-Terraform-Apply

This will apply the infrastructure changes to ENV01 (Production).

This release pipeline will also deploy network resources for the non-prod environments such as env04 and env05. The pipeline is identical to *AIF-NonProd-Terraform-Apply* that update ENV04 environment. The only difference is that it is triggered by the Prod release pipeline and therefore has a different artifact. Follow the instruction as per ENV04 deployment. The resource group for this pipeline is **rg-aif-prod-env01**.

3 Integration Process Flow

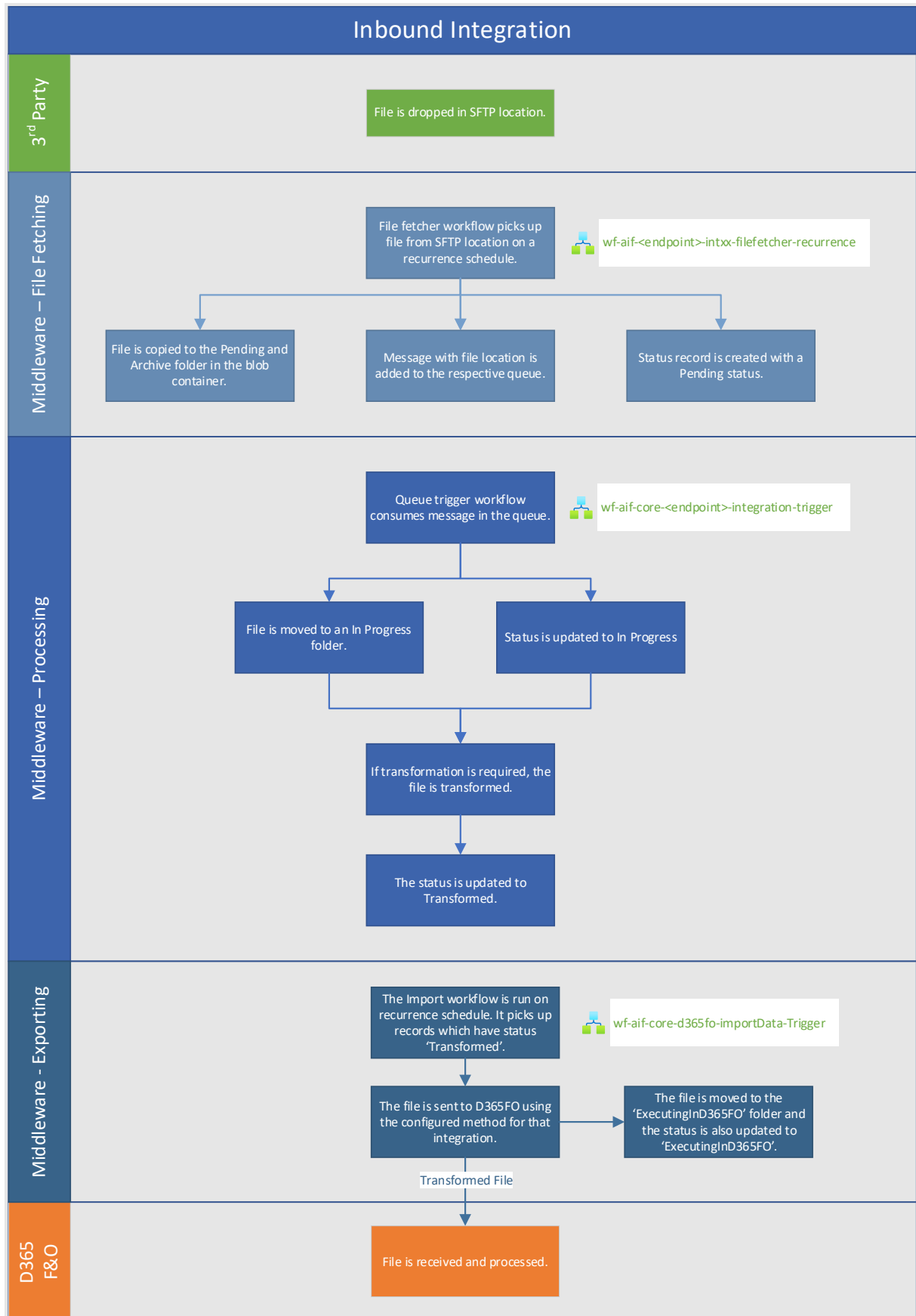
The middleware is split into 4 parts:

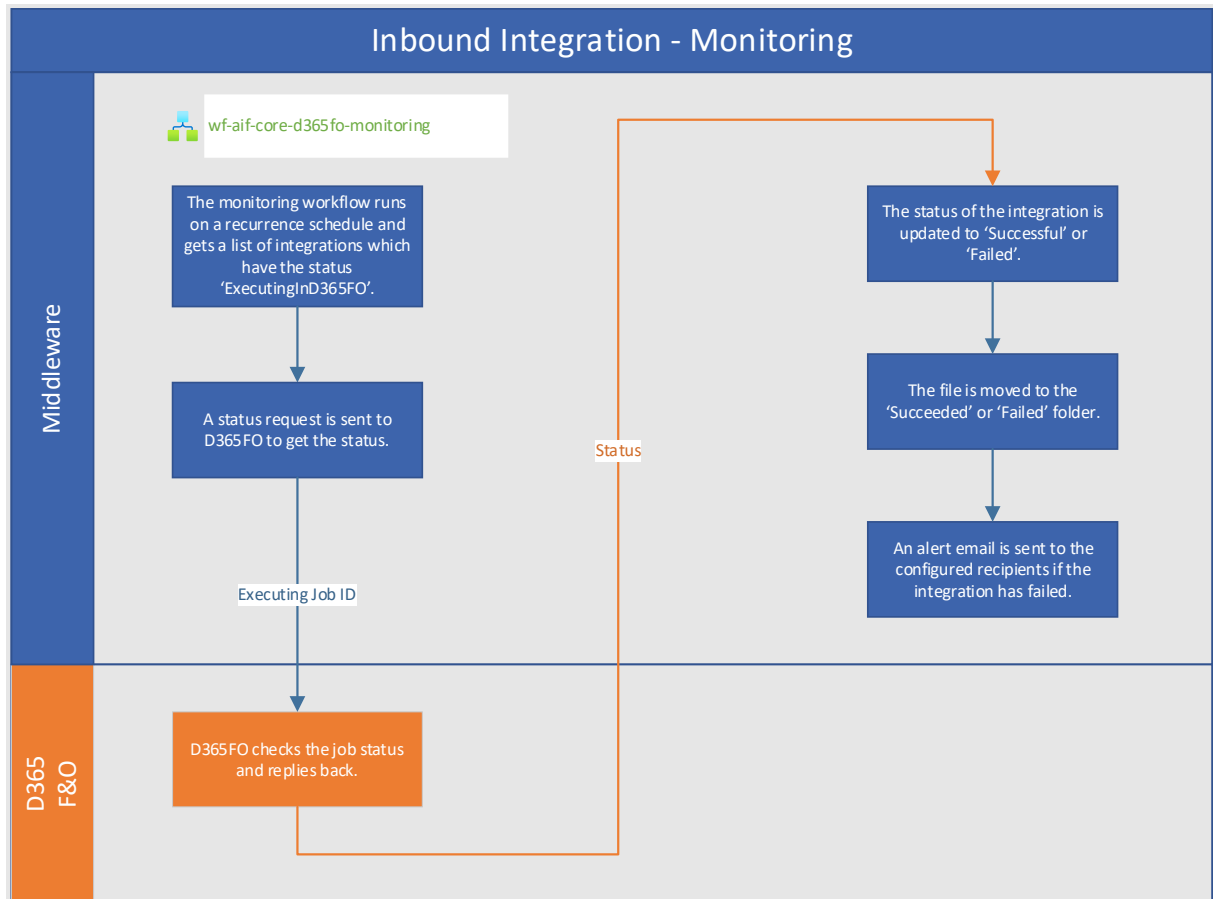
1. File fetching – The process of getting a file from the source.
2. Processing – The process of validating or transforming the file.
3. Exporting – The process of exporting the processed file to the destination.
4. Monitoring – The process of tracking the processing of the file in D365FO.

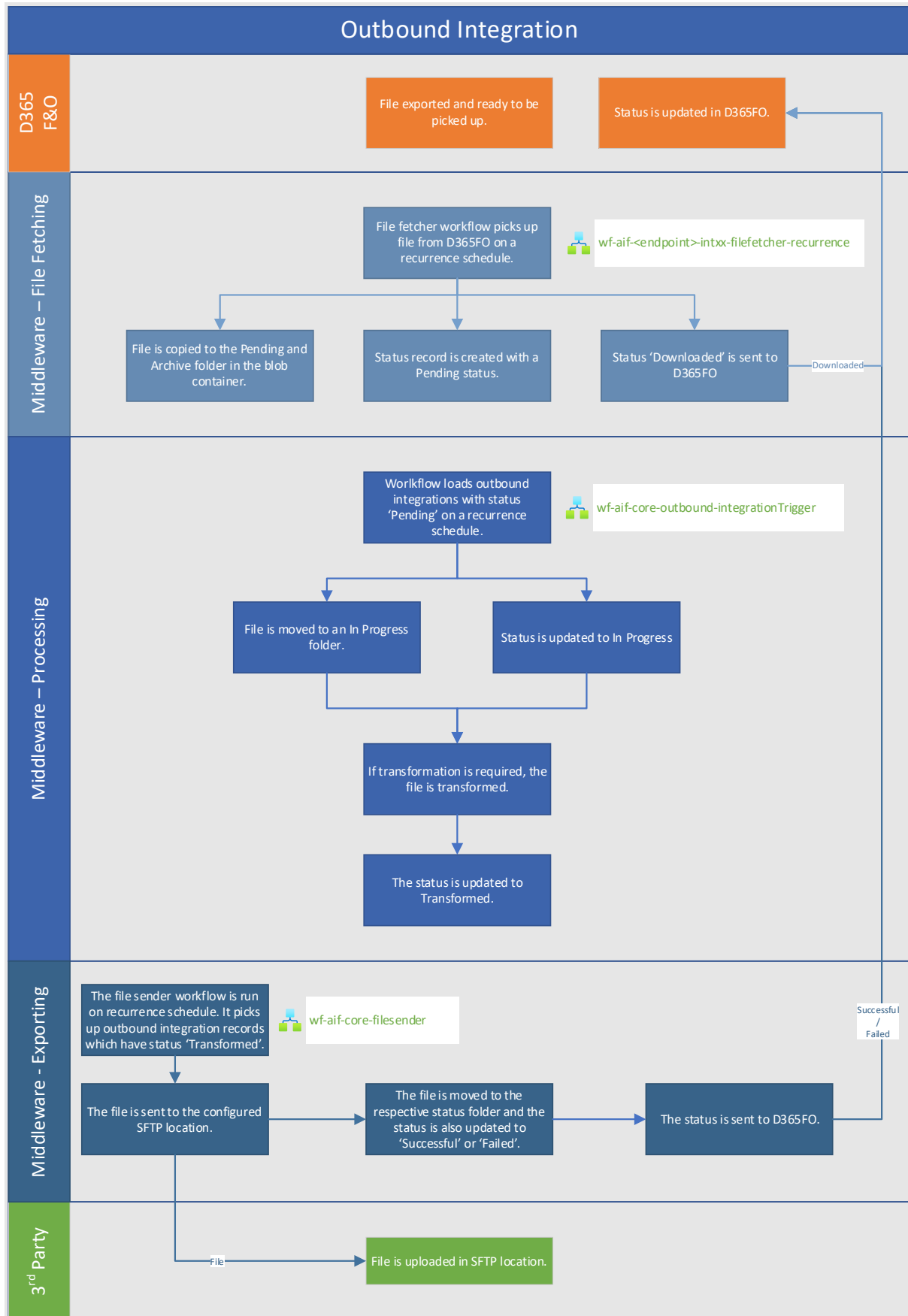
An integration can be either:

- **Inbound** – 3rd party file sent to D365FO.
- **Outbound** – D365FO file exported to a 3rd party.

The following diagrams show a high-level representation of both inbound and outbound integrations.







4 Azure Resource Groups

The following resource groups are set up.

Resource Group	Purpose
rg-aif-nonprod-env03	Contains storage account, logic app and functions used for development purposes.
rg-aif-nonprod-env04	Contains private endpoints, storage account, logic app and functions used for integration testing.
rg-aif-nonprod-env05	Contains private endpoints, storage account, logic app and functions used for UAT.
rg-aif-nonprod-env06	Contains private endpoints, storage account, logic app and functions used for Pre-Production.
rg-aif-nonprod-shared	Contains the keyvault used in the env03/04/05 resource groups.
rg-aif-prod-env01	Contains private endpoints, storage account, logic app and functions used for production.
rg-aif-prod-shared	Contains the keyvault used in the env01 resource groups. Contains other network resources.

Azure resources specific to an environment contain the environment name **envxx** in their name. e.g., the storage account name for env04 is **staaifenv04**.

5 Azure Storage Account

A storage account exists in each environment resource group with the name [staaifenvxx](#). Each storage account contains the following resource types.

5.1 Queues

Queues are used during inbound integrations. Files from third parties are pushed into these queues to be picked up by other workflows.

The following queues are created:

Queue Name	Purpose
sys-aif-inbound-coupa	Stores inbound Coupa files to be processed by the middleware.
sys-aif-inbound-henryschein	Stores inbound HenrySchein files to be processed by the middleware.
sys-aif-inbound-jaggaer	Stores inbound Jaggaer data to be processed by the middleware.
sys-aif-inbound-opuscapita	Stores inbound OpusCapita files to be processed by the middleware.
sys-aif-inbound-datamigration	Stores inbound data migration files. These should not be used for production and were only created for configuring non-prod environments.

5.2 Blob Containers

The Middleware stores files in blob containers while they are being processed. As the status of a file changes, the files are moved to a different folder in the same blob container. Original input files are also permanently archived in blob containers.

The following containers exist:

Container Name	Purpose
sys-aif-inbound-envxx	Contains files which are being imported to D365FO, in their respective status folder. Also contains the original files being imported, before any transformation by the middleware.
sys-aif-outbound-envxx	Contains files which are being exported from D365FO, in their respective status folder. Also contains the original files being exported, before any transformation by the middleware.
dm-aif-datamigration-envxx	Data migration files are dropped here so that they can be processed by the datamigration tool. This should not be used in production.
sys-aif-datamigration-envxx	Contains files which are being imported to D365FO from the data migration tool. The folder structure is the same as the sys-aif-inbound-envxx container.
sys-aif-offline-repository	Container used to mimic SFTP endpoints. When offline mode is used, files are picked up from or delivered to this container. This is not used in production and is used only for development and testing.

5.3 Tables

Tables are used to hold configuration needed by the middleware, as well as store statuses of the integrations. The following tables exist.

5.3.1 configaifD365FOConfigurations

Contains configurations for integrating with the D365FO instance, such as the instance url.

Field	Purpose
PartitionKey	Environment Alias (e.g. cdg.uat)
RowKey	Integration Alias (e.g. INT02)
BaseUrl	The base url of the D365FO instance.
D365FO_FilterCriteria	The filter used in the request url to get files to export from D365FO using OData.
D365FO_ODataUrl	Part of the request url used to get files from D365FO using OData.
DataEntityName	Part of the request url used to get files from D365FO using OData.
DataEntityName_UpdateStatus	Part of the request url used to send status updates to D365FO after a file is exported to a 3 rd party.
IsOutboundFileZipped	Used to identify whether the file exported from D365FO is a zip file.
UseZipFileName	Uses the zip file name instead of the extracted file name. When this is used, it is assumed that the zip file only has one file.
AddTimestampToExtractedFileName	Whether to add the timestamp after extracting a file in a zip file. Only taken into consideration if UzeZipFileName is set to false.

OutboundFileExtension	The file extension of the exported file from D365FO.
DataEntityName_GetErrorMessage	Part of the request url used to get the error message from D365FO after an inbound integration fails in D365FO.
DataEntityName_GetStatus	Part of the request url used to get the status of a file being imported in D365FO.
LegalEntityName	When used, the company is added to D365FO requests with the value in this field.

5.3.2 **configaifD365FORecurringJobs**

Contains configurations for integrating with the D365FO instance using batch jobs.

Field	Purpose
PartitionKey	Environment Alias (e.g. cdg.uat)
RowKey	Integration Alias (e.g. INT02)
RecurringJobId	The recurring job set up in D365FO batch.

5.3.3 configaifEndpointAPIConfigurations

Contains configurations for integrating with an API service, e.g., Jaggaer.

Field	Purpose
PartitionKey	Environment Alias (e.g. cdg.uat)
RowKey	Integration Alias (e.g. INT02)
OAuthURL	The url for OAuth authentication
OAuth_GrantType	The grant type for OAuth authentication
POSTDataEndpointURL	The endpoint url used to send data to the API.

5.3.4 configaifEndpointConfigurations

Contains configurations for integrating with an external service, such as SFTP connection details and paths.

Field	Purpose
PartitionKey	Environment Alias (e.g. cdg.uat)
RowKey	Endpoint Alias (e.g. Coupa)
QueueName	The queue name for the endpoint. Only applicable for 3 rd parties which include inbound integrations.
RequiresSSH	Whether SSH is required to connect to the SFTP. SSH keys are stored in keyvault.

SFTPIntegrationRootPathInbound	The root SFTP path to download files from for inbound integrations.
SFTPIntegrationRootPathOutbound	The root SFTP path to upload files to for outbound integrations.
SFTPRemoteEndpointLocation	An enum identifier for the endpoint. Used by the Azure functions.
SFTPServerURL	The SFTP host url.
SFTPUser	The user used to authenticate to the SFTP. The password is stored in keyvault not in the tables.
localMimicContainerInboundRootPath	The root path to download files from for inbound integrations when using offline mode.
localMimicContainerName	The container name used for offline integrations. (sys-aif-offline-repository)
localMimicContainerOutboundRootPath	The root path to upload files to for outbound integrations when using offline mode.

5.3.5 configaifEndpointConfigurationPaths

Contains endpoint paths for files in specific integrations.

Field	Purpose
PartitionKey	Environment Alias (e.g. cdg.uat)
RowKey	Integration Alias (e.g. INT02)
localMimicStorageIntegrationPath	The path of the folder in the offline container, concatenated with the root paths in the EndpointConfigurations table.
SFTPIntegrationPath	The path of the folder in the SFTP, concatenated with the root paths in the EndpointConfigurations table.
SFTPIntegrationPathMap	Used to identify part of the path, depending on a set of logical keys. This may not be needed for all integrations and can also be handled differently per integration. For Coupa outbound integration INT01, it is being used to determine the target SFTP path, depending on the start of the file name, i.e. the bank.
SFTPArchivePath	If not null, files that are pulled in from an sftp are also moved inside the SFTP to the specified path.

5.3.6 configaifEndpointDownloadConfigurations

Identifies whether to import or delete a file that appears at the endpoint. This can be used in a different way per integration. E.g., for INT34, depending on both the country (FileType1) and the statement type (FileType2), there may be a mixture of actions to be carried out per combination.

Field	Purpose
PartitionKey	Environment Alias
RowKey	Unique row key (e.g. INT02-0). There may be multiple entries per integration.
FileType1	Part of the file name which determines a file type. Each integration can have its own logic of determining FileType1.
FileType2	Part of the file name which determines a file type. Each integration can have its own logic of determining FileType2.
DeleteSFTPFile	Determines whether to delete the file in the SFTP.
DownloadSFTPFile	Determines whether to download the SFTP file.
Description	Used for information purposes. This is not used in any of the functionality.

5.3.7 **configaifEndpointUploadConfigurations**

Identifies different SFTP configurations when uploading files to an SFTP, based on the file type.

Field	Purpose
PartitionKey	Environment Alias
RowKey	Unique row key (e.g. INT02-0). There may be multiple entries per integration.
FileType1	Part of the file name which determines a file type. Each integration can have its own logic of determining FileType1.
SftpUsername	Determines the username used to connect to the SFTP for this file type.
SftpPasswordEnvVarName	Determines the password environment variable name for this file type.
Description	Used for information purposes. This is not used in any of the functionality.

5.3.8 configaiIntegrationActivator

Allows specific integrations to be enabled or disabled.

Field	Purpose
PartitionKey	Environment Alias (e.g. cdg.uat)
RowKey	Integration Alias (e.g. INT02)
Enabled	Determines whether the integration is enabled or not. If an integration is disabled, the starting workflow (filefetcher-recurrence workflows) are triggered but ended in cancelled state.
Remarks	Used for information purposes. This is not used in any of the functionality.

5.3.9 configaiIntegrationConfigurations

Contains configurations needed by the middleware to process an integration, such as blob container paths

Field	Purpose
PartitionKey	Environment Alias (e.g. cdg.uat)
RowKey	Integration Alias (e.g. INT02)
AIFArchivedOriginalSourceRootFilePath	The integration specific archive location used to store the original input files.
AIFLogsPath	The integration specific logs location.
AIFOutboundContainerName	The container name used for outbound integrations.

AIFOutboundRootFolderPath	The location used as the starting location of outbound integration files.
AIFStorageAccountPublicURL	The public url of the storage account. Used in alert emails.
AIFThirdPartyIntegrationReference	Used for information purposes. This is not used in any of the functionality.
D365FOAPI	The way an export or an import in D365FO is implemented. http/enqueue/dequeue/default.
IntegrationDirection	Inbound or Outbound
IntegrationType	The 3 rd party name. e.g. Coupa
AIFInboundContainerName	The container name used for inbound integrations.
AIFInboundQueueName	The queue name used for inbound integrations. Endpoint specific.
AIFInboundRootFolderPath	The location used as the starting location of inbound integration files.
AIFInboundSecondaryContainerName	Obsolete. Field used for specific integration which is now obsolete.

5.3.10 configaiIntegrationFileExclusions

Used to exclude files from being exported when part of a zip file.

Field	Purpose
PartitionKey	Environment Alias (e.g., cdg.uat)
RowKey	Integration Alias (e.g. INT02)
ExcludedFiles	Files to exclude when exporting files. This is used when using the dequeue method, which returns a zip file with two metadata files which need to be excluded from the integration.

5.3.11 configaiIntegrationMappings

The scope of this table is to map the fields from the Third Party field schema to D365FO field schema. This was specifically designed for Jaggaer.

Field	Purpose
PartitionKey	Its a unique identifier which defines the fieldName..
RowKey	Integration Alias (e.g., INT02)
IntegrationMappingField	The D365FO field name which maps the Third-Party Mapping Field Value.
ThirdPartyMappingFieldValue	The Third Party field name which maps with the D365FO field value.

5.3.12 configaifSourceFileConfigurations

Contains configurations related to the file name of the file being imported/exported.

Field	Purpose
PartitionKey	Environment Alias (e.g. cdg.uat)
RowKey	Endpoint Alias (e.g. Coupa)
FileExtension	The file extension expected from the endpoint.
FileNameDelimiter	The delimiter of the filename, sometimes used to get the logical type of a file.
FileTypeSupStringLocation	The location of the logical file type when the file name is split using the FileNameDelimiter.

5.3.13 sysaifIntegrationStatusTracker

Contains the status of a file being imported/exported, including its current location in the blob container.

Field	Purpose
PartitionKey	Environment Alias (e.g. cdg.uat)
RowKey	The original file name.
AIFBlobContainer_IntegrationSourceFileLocation	The container name of the original source file location.
AIFIntegrationDirection	Inbound or Outbound

	The status of the integration.
AIFIntegrationStatus	Pending / InProgress / Transformed / ExecutingInD365FO / Failed / Successful.
AIFStorageAccountPublicURL	The storage account public url. Used for alert emails.
IntegrationAlias	The integration alias (e.g. INT02)
IntegrationFileName	The integration file name, excluding the extension.
IntegrationFolderName	The folder name which contains the file, usually a representation of the integration start date.
IntegrationOriginalSourceFileLocation	The full path of the original integration file. If the original file was a zip file and it was extracted, the path here is for the extracted file.
IntegrationOriginalSourceZipFileLocation	The full path of the original zip file, if the original file was a zip file which was extracted.
IntegrationSourceFileExtension	The file extension.
IntegrationSourceFileLocation	The file path in its current state. e.g. If the status is InProgress, it will be in an InProgress folder.
IntegrationType	The endpoint name, e.g. Coupa.
RetryCounter	Obsolete. Set to 0.
D365FOExecutionId	The execution ID in D365FO, related to this integration.

RelatedTo

Used in specific integrations where integration files are derived from, or related to another file.

6 Azure Logic App Standard

Logic app [la-aif-envxx-integration](#) app includes workflows that dictate the entire middleware process, from picking up the file, transforming it and sending it to the destination.

6.1 Inbound Integration Workflows

The following is a list of workflows in the logic app, used for inbound integrations:

Workflow	Trigger	Purpose
wf-aif-<endpoint>-int<xx>-filefetcher-recurrence	Recurrence	The start of an inbound integration. An instance exists for every endpoint/integration alias combination. Checks if the integration is enabled in the <code>configaifIntegrationActivator</code> table and calls the <code>wf-aif-core-filefetcher</code> workflow if it is.
wf-aif-core-filefetcher	Http	Fetches files from a 3rd party, adds them to the respective endpoint queue and updates the status of the integration to Pending.
wf-aif-core-<endpoint>-integrationTrigger	Queue	Starts the processing of an imported file. Updates the integration status to InProgress and calls the respective integration workflow.
wf-aif-<endpoint>-int<xx>-integration-<logical name>	Http	An instance exists for every endpoint/integration alias combination. Processes the file by transforming it if required. Updates the status to Transformed.
wf-aif-core-d365fo-importData-Trigger	Recurrence	Starts the delivery of the file to D365FO.

wf-aif-core-d365fo-importData	Http	Imports the file in D365FO using the configured method. Updates the status to ExecutingInD365FO.
wf-aif-core-d365fo-monitoring	Recurrence	Checks if integrations with status ExecutingInD365FO have been successfully imported in D365FO. Updates the status to Successful or Failed if the files have been processed in D365FO. Sends alert emails if the file import failed.

6.2 Outbound Integration Workflows

The following is a list of workflows in the logic app, used for outbound integrations.

Workflow	Trigger	Purpose
		The start of an outbound integration. An instance exists for every endpoint/integration alias combination.
wf-aif-<endpoint>-int<xx>-filefetcher-recurrence	Recurrence	Checks if the integration is enabled in the configaifIntegrationActivator table and calls the wf-aif-core-d365fo-exportData workflow if it is.
wf-aif-core-d365fo-exportData	Http	Retrieves files from D365FO using the configured method. Adds status records in Pending state.
wf-aif-core-d365fo-handleDownloadedFile	Http	Part of the exportData workflow to simplify the workflow.
wf-aif-core-outbound-integrationTrigger	Recurrence	Starts the processing of an exported file. Updates the integration status to InProgress and calls the respective integration workflow.
wf-aif-<endpoint>-int<xx>-integration-<logical name>	Http	An instance exists for every endpoint/integration alias combination. Processes the file by transforming it if required. Updates the status to Transformed
wf-aif-core-filesender	Recurrence	Sends the transformed file to the 3 rd party (e.g. SFTP).
wf-aif-core-filesender-postsend	Http	Part of the filesender workflow. Used to update status of an outbound integration in D365FO, after the file has been delivered to the 3 rd party.
wf-aif-jaggaer-api-datasender	Recurrence	Sends the transformed data to the Jaggaer API.

6.3 Shared Workflows

The following is a list of workflows in the logic app, used in both inbound and outbound integrations.

Workflow	Trigger	Purpose
wf-aif-core-integrationStatusUpdate	Http	Used to update the status of an integration in the sysaifIntegrationStatusTracker table and move the file to the correct folder in the blob container.

6.4 Logic App Parameters

The following is a list of parameters used during the execution of the workflows mentioned above.

Parameter	Purpose
D365FOAADTenantId	The tenant ID used to authenticate with D365FO.
D365FOAppRegClientId	The client ID used to authenticate with D365FO.
D365FOAppRegClientSecret	The client secret used to authenticate with D365FO. Retrieved from app settings which retrieves the value from key vault.
blob.ExecutingInD365FOPath	The blob path for blobs in status 'ExecutingInD365FO'.
blob.FailedPath	The blob path for blobs in status 'Failed'.
blob.InProgressPath	The blob path for blobs in status 'InProgress'.
blob.PendingPath	The blob path for blobs in status 'Pending'.
blob.SuccessfulPath	The blob path for blobs in status 'Successful'.
coupa.integrationTypeAlias	The integration type name for Coupa.

dm.integrationTypeAlias	The integration type name for Data Migration.
exquise.integrationTypeAlias	The integration type name for Exquise.
global.INT14CreditEmailAlert	The email used for INT14 to send alerts when the file is of type 'D' (i.e., Debit). Lately in the project, CDG decommissioned INT14.
global.MonitoringEmailAlert	The email address used as a recipient for the monitoring alert email.
global.environmentAlias	The environment alias, e.g., cdg.uat or cdg.prod.
global.integrationDirection-inbound	The name of the integration direction for inbound integrations.
global.integrationDirection-outbound	The name of the integration direction for outbound integrations.
global.useOnlyBuildInConnectors	Determines whether to use only the Built-In connectors or not. By default, this should be set to ' <i>false</i> '.
global.useSFTPEndpoints	Determines whether to use SFTP endpoints or the offline containers. True: Use real SFTP endpoints. False: Use the offline container instead.
henryschein.integrationTypeAlias	The integration type name for Henry Schein.
jaggaer.APIEndpoint_ClientID	The Client ID used for Jaggaer authentication. Retrieved from app settings, keyvault.
jaggaer.APIEndpoint_ClientSecret	The Client Secret used for Jaggaer authentication. Retrieved from app settings, keyvault.
jaggaer.integrationTypeAlias	The integration type name for Jaggaer.

mercur.integrationTypeAlias	The integration type name for Mercur.
opuscapita.integrationTypeAlias	The integration type name for OpusCapita.
pms.CDG_SFTP_PMS_Password	The SFTP password for PMS. Retrieved from app settings, keyvault.
pms.integrationTypeAlias	The integration type name for PMS Frenda.
status.ProcessCancelled	The name for status 'Cancelled'.
status.ProcessDeleted	The name for status 'Deleted'.
status.ProcessExecuting	The name for status 'ExecutingInD365FO'.
status.ProcessFailed	The name for status 'Failed'.
status.ProcessFileTransformed	The name for status 'Transformed'.
status.ProcessInProgress	The name for status 'InProgress'.
status.ProcessPending	The name for status 'Pending'.
status.ProcessSucceeded	The name for status 'Successful'.
successfactors.IntegrationTypeAlias	The integration type name for SuccessFactors.
tagetik.IntegrationTypeAlias	The integration type name for Tagetik.

7 Azure Functions

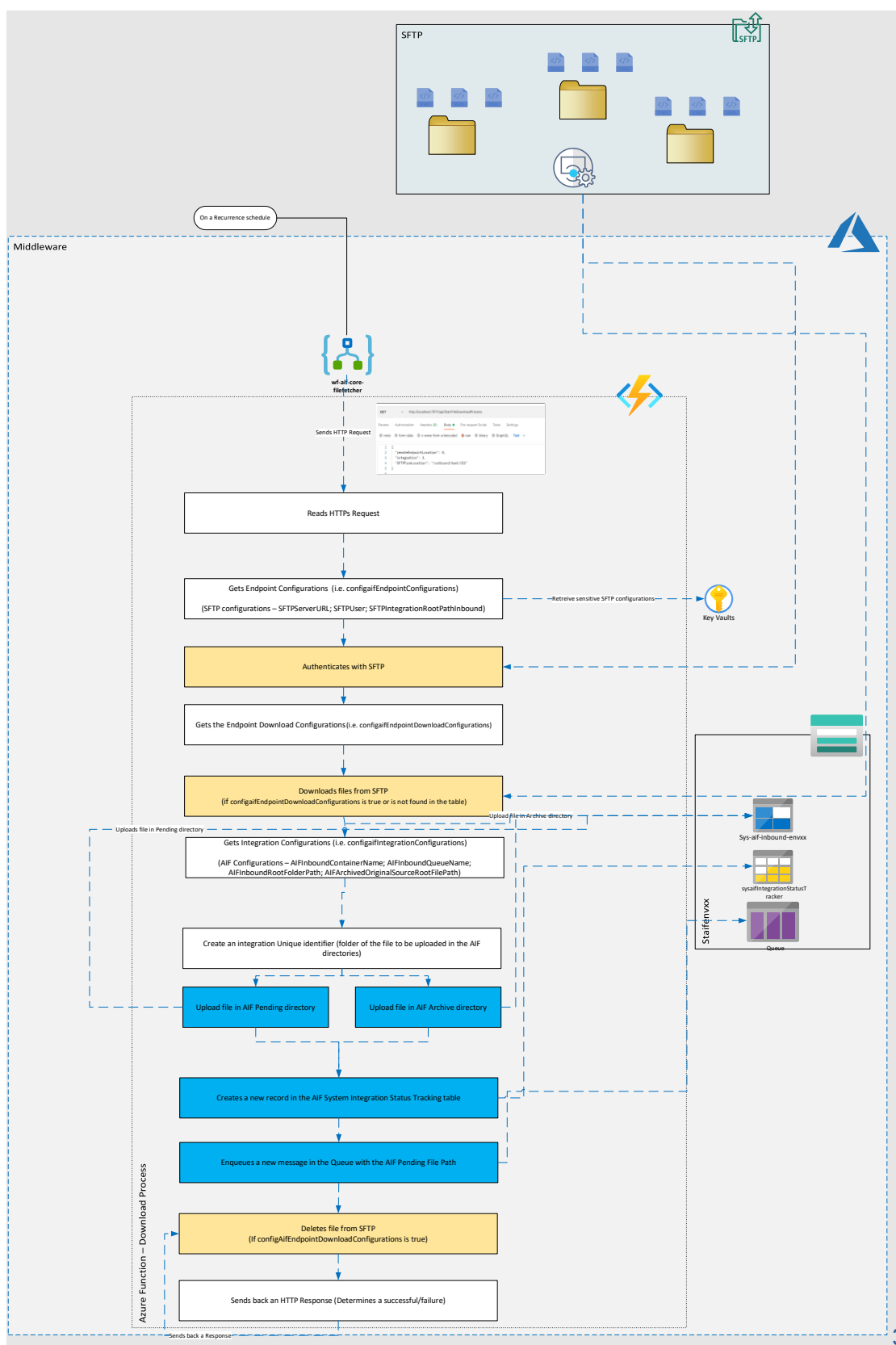
Function app [func-aif-envxx-integration](#) includes the following functions.

Function	Purpose
StartFileDownloadProcess	Downloads a file from an external endpoint (e.g. SFTP)
StartFileUploadProcess	Uploads a file to an external endpoint.(e.g. SFTP).
StartInboundIntegrationProcess	Transforms a file from a 3 rd party before being imported to D365FO
StartOutboundIntegrationProcess	Transforms a file from D365FO before being exported to a 3 rd party.

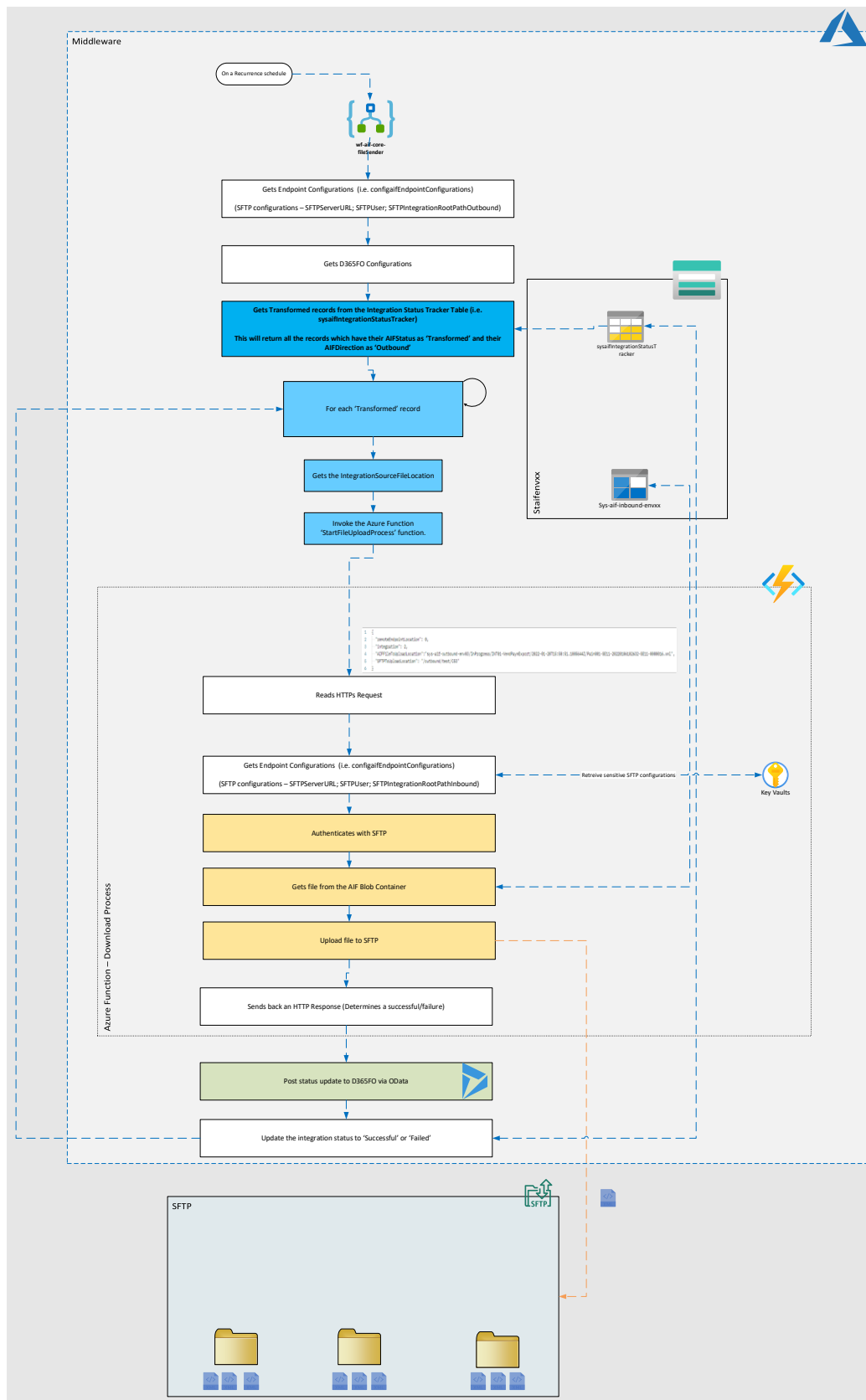
Function app [func-aif-envxx-procurementAPI](#) includes the following functions.

Function	Purpose
ProcessProcurement	Its an API which authenticates, consumes the HTTP Post, transforms it into a file and uploads it to the middleware storage account for processing.

8 File Download Process flow

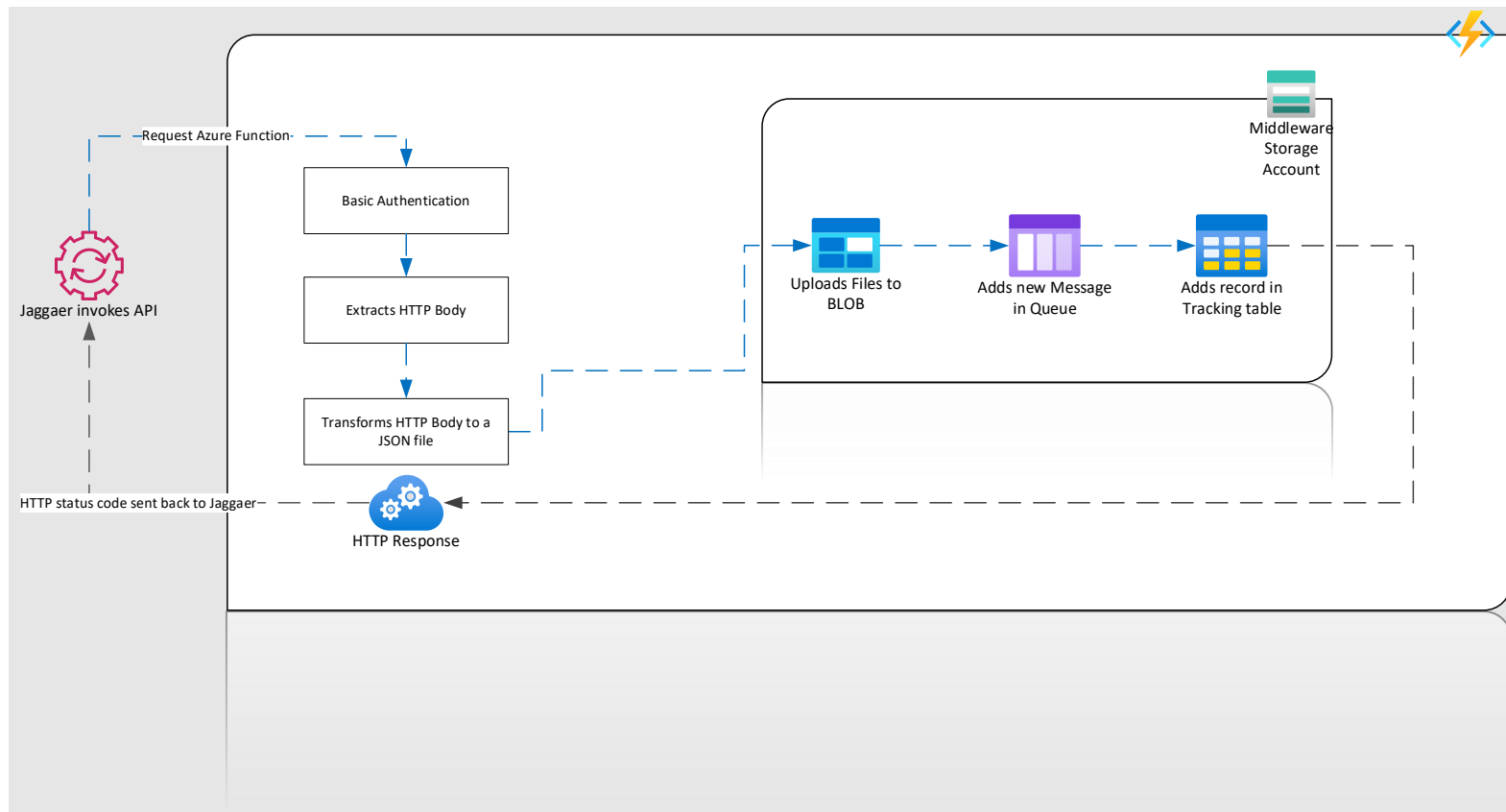


9 File Upload Process flow



10 Procurement API Process flow

The Procurement API was specifically designed as per the Jaggaer third party requirements in order for the third party to invoke the API. Once invoked, the API will verify the authentication, extracts the HTTP Body and transforms it into a JSON file. The JSON file is then uploaded as a BLOB to the storage account blob container, added to the Jaggaer queue and logs the entry in the middleware status tracker table.



11 D365FO status monitoring

11.1 Inbound Integration Status Monitoring

For integrations coming from a 3rd party into D365FO, the data will go through the middleware before going into D365FO. If something wrong happens during transition, before the data manages to reach D365FO, D365FO will not have any details of this integration. The status in this case will only be available in the middleware.

Once the data reaches D365FO, D365FO will import the data. In this case, if the import fails because of a data issue or an incorrect format of the file, the **failed** or **processed with error** status will show in D365FO.

Processing status for recurring data job					
Standard view ▾					
Filter					
Message id	Message status	Batch status	Process message in this c...	Submitted by	Submitted date time
{570277C8-C0C9-49E0-8726-62622F46D3E3}	Processed with err...	SE11		Admin	3/9/2023 2:21:13 PM
{6C70906A-6FFB-4ED2-8E81-4C9DE3516DCA}	Processed	FI11		Admin	2/21/2023 1:20:44 PM
{1B08DEE1-1C3E-48C8-AA25-7C4369AD0F5A}	Processed	FI11		Admin	2/21/2023 10:15:36 AM
{98FB8407-A59B-4D79-93B6-CA3E220DE6B3}	Processed	FI11		Admin	2/21/2023 10:15:35 AM
{7D11E8D4-9AA4-4A70-9066-341F30F7F8CF}	Processed	FI11		Admin	2/21/2023 10:15:35 AM
{2284C24F-2EC6-4792-B627-67C389F7066F}	Processed	FI11		Admin	2/20/2023 5:53:27 AM
{1F07F95C-86E9-4416-8630-11C8C91315DC}	Processed	FI11		Admin	2/20/2023 5:53:27 AM
{B2BA1568-E579-459C-B043-17CE641A2C7F}	Processed	FI11		Admin	2/17/2023 12:35:18 PM
{415E4FE1-C9C8-4D23-AECB-40E1628A348E}	Processed	FI11		Admin	2/17/2023 12:35:17 PM
{10958672-023E-47DB-B156-CE2E17EFA97C}	Processed	FI11		Admin	2/17/2023 9:31:21 AM
{AB4CE3E1-5A04-44EA-9EF5-1BAE70AEFCE0}	Processed with err...	FI11		Admin	2/17/2023 9:31:21 AM
{7221CA7B-B311-411E-BAAD-10E5CD7F581A}	Processed	FI15		Admin	2/15/2023 9:39:41 AM
{3D609A2D-5203-4B7B-A7A6-30473E4B003E}	Processed	FI15		Admin	2/13/2023 11:39:49 AM
{F4273D6F-5EC5-4148-A207-2840BD291B6B}	Processed	FI11		Admin	2/13/2023 9:21:08 AM
{E3D30F03-F552-4B53-8FF8-C223DA255A54}	Processed	FI11		Admin	2/13/2023 9:21:07 AM
{BF4A8CA5-008A-479B-88A9-37B91AD38DD6}	Processed	FI11		Admin	2/13/2023 9:21:07 AM

11.2 Outbound Integration Status Monitoring

For integrations coming from D365FO into a 3rd party system, the status in D365FO will be updated from the start of the integration. Depending on the integration, the status shown could be one of the following

- **Ready** – Ready for pickup by the middleware.
- **Downloaded** – Picked up and being processed by the middleware.
- **Exported** – Successfully exported to the 3rd party SFTP.

- **Failed** – Failed to export to 3rd party.
- **Processed** – In case of INT32, processed means that the file is ready for pick up.
- **Acknowledged** – In case of INT32, the acknowledged flag shows that the file was downloaded successfully by the middleware.

Log for the export format

Standard view * ▼

Filter

Batch Id	Journal batch number	Created by	Created date and time	Status	File name	Method of pay
(68501DC3-E89B-4753-8C45-6B...	SE11-JB00001184	pavanvattathil.ravi	3/29/2023 10:26:56 AM	Ready	DNB-OCR-20230329102639-SE...	OCRPayment
(09A772B9-1A8A-4FA9-9B4F-25...	SE11-JB00001184	pavanvattathil.ravi	3/29/2023 10:26:21 AM	Ready	DNB-Advice-20230329102602-S...	Advice
(E4A9906B-2420-4869-8E35-1B...	SE11-JB00001265	tanyaverma3	3/27/2023 10:44:38 AM	Ready	DNB-OCR-20230327104318-SE...	OCRPayment
(3116C3AF-5DA5-4D5F-96EA-6...	SE11-JB00001265	tanyaverma3	3/27/2023 5:27:12 AM	Ready	DNB-OCR-20230327052627-SE...	OCRPayment
(46596349-AC2D-4698-B1AD-E...	FI11-JB00000012	tomas.monte	2/21/2023 10:30:10 AM	Ready	DNB-FI11-SEPA-2023022112294...	SEPA
(E7E125E6-2EB0-448A-AD72-87...	FI11-JB00000016	tomas.monte	2/21/2023 6:49:37 AM	Ready	DNB-FI11-SEPA-2023022108492...	SEPA
(06A8E9F2-E2AA-4FC6-86F1-87...	FI11-JB00000010	tanyaverma3	2/17/2023 1:57:34 PM	Ready	DNB-FI11-SEPA-2023021713571...	SEPA
(05B5ED88-CF15-4B84-B6D4-F8...	FI15-JB00000011	tomas.monte	2/15/2023 9:48:20 AM	Exported	DNB-FI15-SEPA-2023021511475...	SEPA
(C1FDF23F-1045-40B5-A1B5-23...	FI11-JB00000007	tanyaverma3	2/13/2023 1:45:39 PM	Exported	DNB-FI11-SEPA-2023021313452...	SEPA
(B9061677-1014-477C-8B97-66...	FI11-JB00000007	tomas.monte	2/13/2023 1:03:26 PM	Exported	DNB-FI11-SEPA-2023021315030...	SEPA
(EF688FF9-3C54-404A-8541-4B...	FI15-JB00000008	tomas.monte	2/13/2023 11:47:57 AM	Exported	DNB-FI15-SEPA-2023021313471...	SEPA
(68C53C1F-16CA-4797-8714-35...	SE11-JB00001179	Jelena.Kovacevic	2/2/2023 2:58:00 PM	Exported	DNB-OCR-20230202155744-SE...	OCRPayment
(01826C7C-6BC1-4B70-AD1A-2...	SE15-JB00000120	Jelena.Kovacevic	2/2/2023 12:48:09 PM	Exported	DNB-Advice-20230202134757-S...	Advice
(376ABCEB-8F0F-49CA-8E69-B6...	SE14-JB00000207	Jelena.Kovacevic	2/2/2023 12:45:54 PM	Exported	DNB-Advice-20230202134538-S...	Advice

Log

File pushed to Coupa

Processing status for recurring data job

Standard view ▼

Filter

Message id	Message status	Batch status	Total records e...	File uploaded s...	Processing started date time
(B1DD9235-026A-4CF7-8F0B-73F8FD9F8D28)	Processed		3136	✓	3/29/2023 10:22:29 AM
(87594205-42B9-4466-B496-66CE12021D94)	Acknowledged		3136	✓	3/23/2023 1:48:38 PM
(30101361-B896-47B0-A50C-01D2DC6B348A)	Acknowledged		3136	✓	3/23/2023 1:46:41 PM
(5795FCF0-72AB-457C-8D45-80F75163C399)	Acknowledged		3136	✓	3/23/2023 12:59:37 PM

12 Monitoring

12.1 Status Tracker Table

The *sysaifIntegrationStatusTracker* table can be used to track the status of an integration. The query filters can help narrowing down on a specific integration and this table will show useful information such as the status, file location and any job execution ID in D365FO.

sysaifIntegrationStatusTracker

Close Query

Import

Export

Add

Edit

Select All

Column Options

Delete

Table Statistics

Refresh

And/Or

Field

Type

Operator

Value

+ X

Timestamp

DateTime

>=

Last 7 days

+ X

And

IntegrationAlias

String

=

INT01

+ Add new clause

Advanced Options

PartitionKey

RowKey

Timestamp

AlfIntegrationStatus

IntegrationAlias

cdg.uat

DNB-SE11-Advice-20230119105328-SE11-J800000241

2023-01-19T11:08:11.2216073Z

Successful

INT01

cdg.uat

ISO20022 Credit transfer_181620230116551655-2023-01-18T13-17-57.8847559Z

2023-01-18T13:20:49.0000442Z

Successful

INT01

Note that the row key in this table is represented by the file name. If a failed file is retried without being renamed, the status record will be overwritten.

The following is a list of statuses and their respective folder location.

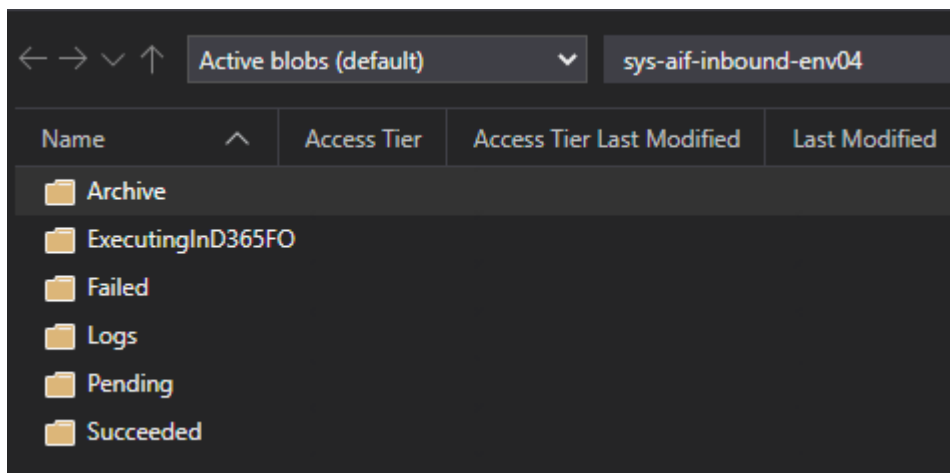
Status	Folder	Description
Pending	Pending	The file is picked up from the source system and is waiting to be processed by the middleware.
InProgress	InProgress	The pending file is picked up for processing.
Transformed	InProgress	The file is optionally transformed or ready from processing. This is the version of the file to be sent to the destination.
ExecutingInD365FO	ExecutingInD365FO	The inbound file is sent to D365FO and is processing in D365FO.

Successful	Succeeded	For inbound, the file is successfully processed in D365FO. For Outbound, the file is successfully exported to the 3 rd party.
Failed	Failed	There was a failure during any part of the integration process.

The folders can be found under the respective blob container. For inbound integrations that is **sys-aif-inbound-envxx** and for outbound it is **sys-aif-outbound-envxx**.

Apart from the statuses, there is:

- an **Archive** folder, which contains all source files before they are processed by the middleware
- a **Logs** folder, which contains function logs related to the integrations.



12.2 Azure Function Logs

Azure functions used during the integration of a file also log data in the respective blob container. E.g., for inbound integrations it will be in:

sys-aif-inbound-envxx/Logs/<Integration Identifier>/<Timestamp Folder>

Alternatively, logs can also be checked in Application Insights within the Azure portal:

1. Go to the application insights resource of the specific environment, e.g. app-aif-env05
2. Go to Logs and type the following query.

```
traces
| where message contains '[AIF_'
| extend trimmedMessage = substring(message, 10)
| extend category = customDimensions["Category"]
| extend requestId = operation_ParentId
//| where requestId == '7eff30b846f51b4b'
| project timestamp, requestId, trimmedMessage, category, customDimensions
```

The above query will search for log messages coming out of Azure functions. Once you identify a log message from a specific request, you can also filter by *requestId*, which will give you the log messages of a specific request.

Run

Time range : Last 24 hours

Save

Share

New alert rule

Export

Pin

```

1 traces
2 | where message contains '[AIF_'
3 | extend trimmedMessage = substring(message, 10)
4 | extend category = customDimensions["Category"]
5 | extend requestId = operation_ParentId
6 | where requestId == '7eff30b846f51b4b'
7 | project timestamp, requestId, trimmedMessage, category, customDimensions

```

ResultsChart

timestamp [UTC] ↑↓	requestId	trimmedMessage	category
> 1/19/2023, 1:49:57.586 PM	d86d0df668bc0e2a	DownloadFile - Download SFTP file complete	INT33
> 1/19/2023, 1:49:57.586 PM	d86d0df668bc0e2a	DownloadFromSftp - Downloaded files from SFTP directory.	INT33
> 1/19/2023, 1:49:56.888 PM	d86d0df668bc0e2a	DownloadFromSftp - Downloading files from SFTP directory.	INT33
> 1/19/2023, 1:49:56.888 PM	d86d0df668bc0e2a	DownloadFromSftp - Connection to SFTP established.	INT33
> 1/19/2023, 1:49:55.794 PM	d86d0df668bc0e2a	DownloadFromSftp - Attempting to connect to SFTP...	INT33
> 1/19/2023, 1:49:55.769 PM	d86d0df668bc0e2a	GetEndPointDownloadConfigurations - Getting end point d...	INT33
> 1/19/2023, 1:49:55.726 PM	d86d0df668bc0e2a	GetEndPointConfigurationsPaths - Getting end point integr...	INT33
> 1/19/2023, 1:49:55.688 PM	d86d0df668bc0e2a	GetEndPointConfigurations - Getting end point integration ...	INT33
> 1/19/2023, 1:49:55.638 PM	d86d0df668bc0e2a	GetIntegrationConfigurations - Getting integration configur...	INT33
> 1/19/2023, 1:49:55.637 PM	d86d0df668bc0e2a	DownloadFile - Download SFTP file started	INT33

12.3 Application Insights

12.3.1 Scope of Application Insights

Application Insights is an extendable feature of Azure Monitor and it provides Application Performance Monitoring (i.e. also known as APM) features on an application. APM contains a number of tools which are useful to monitor applications in all phases of the project.

APM offers two type of ways to monitor data:

- Real-Time visualisation of your application wherein you can monitor real-time metrics on your deployed application and;
- Reactively review application execution data to troubleshoot the cause of an incident.

12.3.1.1 Dashboards

A standard dashboard was created for CDG as part of the Monitoring solution to be used as a tool to monitor the Production middleware environment.

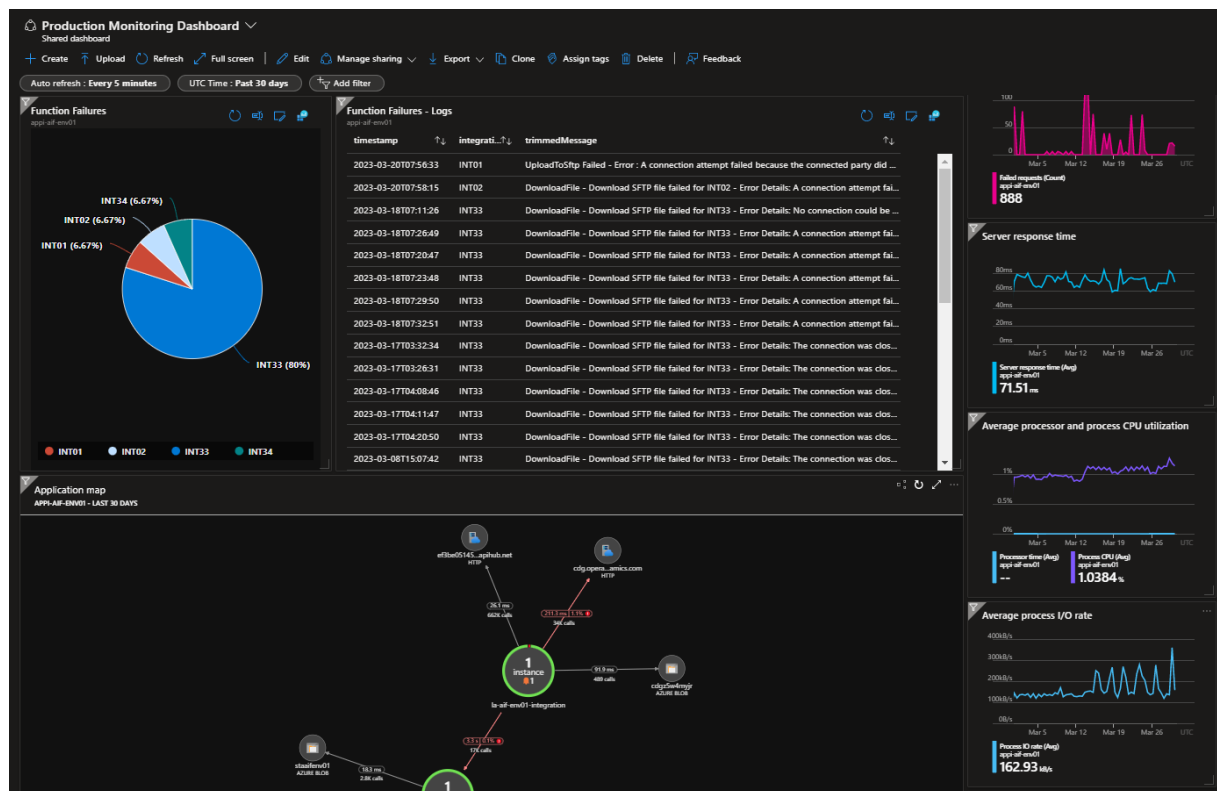


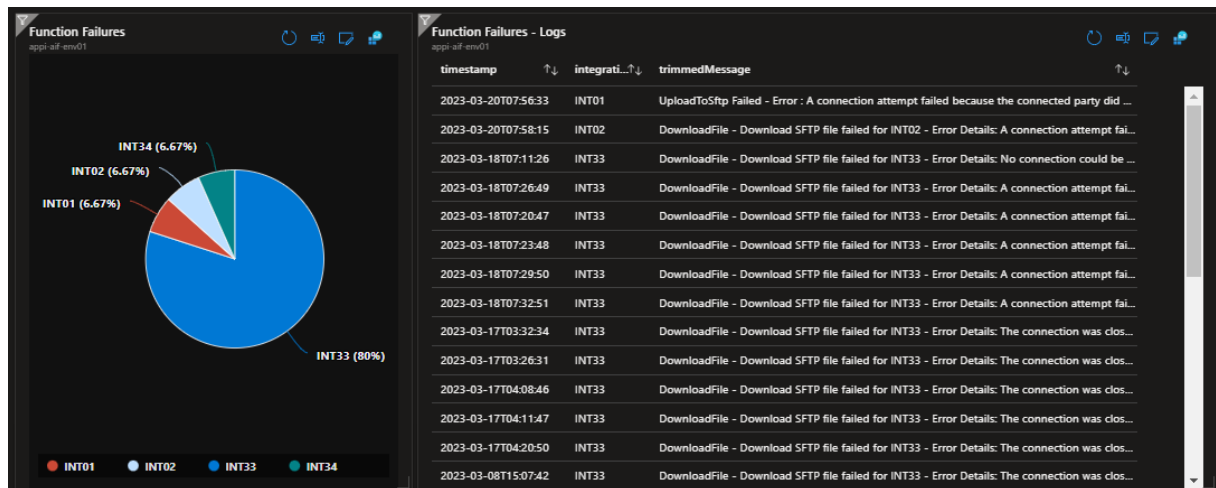
Fig: Shows a screenshot of the Production Monitoring Dashboard.

Note: The Production dashboard display name is **Production Monitoring Dashboard**

In the above dashboard, there are several tiles which gives the BAU user the error logs of the chosen time frame.

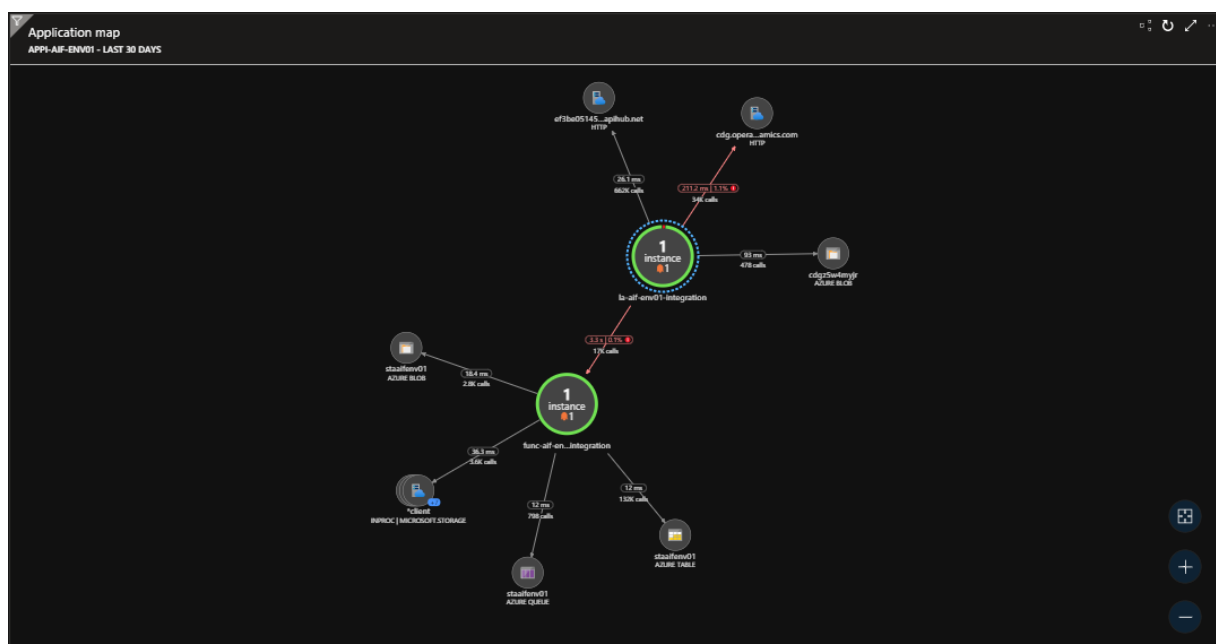
Function Failures:

These two tiles are placed near each other which gives a visualisation of the errors per each integration and a list of error logs per each integration.



Application Map:

An application map is included in the dashboard, demonstrating the communication amongst the resources. This map demonstrates all the connections (requests and responses) that are being made. The map also demonstrates the average calls from the azure functions to its dependencies.



Infrastructure Metrics:

Additionally, to the above, this dashboard contains a list of infrastructure related tiles which visualises the availability and failures experienced in the Production environment.

Failed Requests:

This tile visualises the number of Azure failures on the Production environment.

Server response time:

This tile visualises the performance details for the different operations in your application.

By identifying the operations with the longest duration, you can diagnose potential problems or target your ongoing development to improve the overall performance of the application.

Average processor and process CPU utilization:

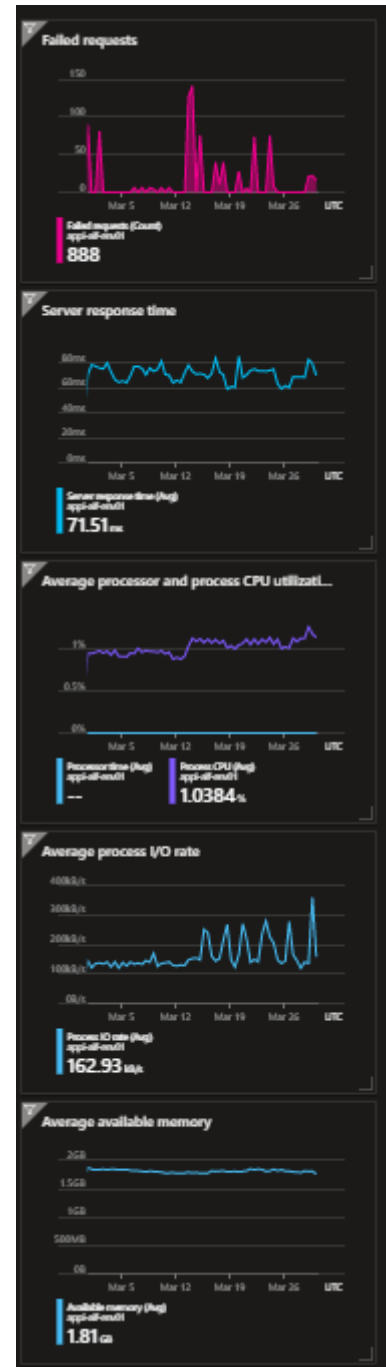
This tile visualises the average CPU utilisation that the middleware is utilising to process the integration operations.

Average process I/O rate:

This tile visualises how many process IO Bytes are being processed per second.

Average available memory:

This tile visualises the current available memory being utilised by the Production Azure environment.



12.3.1.2 Alerts

12.3.1.3 Scope of alerts

In Azure, we can subscribe to an alert so if a failure happens on the middleware resources, an alert will be triggered.

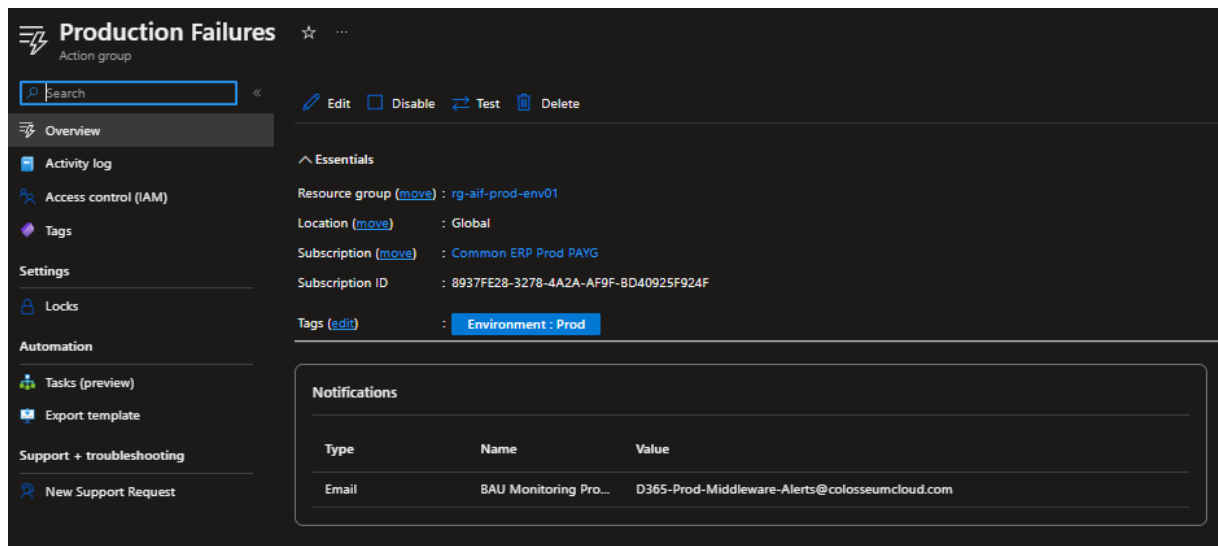
A notification will be sent to the selected recipients of an Action Group.

12.3.1.4 Action Group

An Action Group is a set of configurations that may be used in several alerts. In the action group, few recipients are configured to receive alerts once an alert is triggered by a specific rule.

The configured action group may be used with every alert rule that is applied.

On the Production Application Insights, the following Action Group was created:



The screenshot shows the 'Production Failures' Action Group configuration in the Azure Portal. The left sidebar contains navigation links: Overview, Activity log, Access control (IAM), Tags, Settings, Locks, Automation, Tasks (preview), Export template, Support + troubleshooting, and New Support Request. The main content area is divided into 'Essentials' and 'Notifications' sections.

Essentials

- Resource group (move): rg-aif-prod-env01
- Location (move): Global
- Subscription (move): Common ERP Prod PAYG
- Subscription ID: 8937FE28-3278-4A2A-AF9F-BD40925F924F
- Tags (edit): Environment : Prod

Notifications

Type	Name	Value
Email	BAU Monitoring Pro...	D365-Prod-Middleware-Alerts@colosseumcloud.com

12.3.1.5 Alert Rule

The Alert rules are configurations that defines the threshold of the signal prior triggering an alert.

Create an alert rule

ScopeConditionActionsDetailsTagsReview + create

Configure when the alert rule should trigger by selecting a signal and defining its logic.

Signal name

Failed requests

[See all signals](#)

Alert logic

We have set the condition configuration automatically based on popular settings for this metric. Please review and make changes as needed.

Threshold Static Dynamic

Aggregation type Count

Operator Greater than

Unit Count

Threshold value 0

Split by dimensions

Use dimensions to monitor specific time series and provide context to the fired alert. Dimensions can be either number or string columns. If you select more than one dimension value, each time series that results from the combination will trigger its own alert and will be charged separately. [About monitoring multiple time series](#)

Dimension name	Operator	Dimension values	Include all future values
Select dimension	=	0 selected	☐
Add custom value			

When to evaluate

Check every 5 minutes

Lookback period 5 minutes

[+ Add condition](#)

Preview \$0.10 USD/month

Whenever the count Failed requests is greater than 0

Time range : Over the last 6 hours Time series : Aggregate

Failed requests (Count)
over the last 6 hours

14

12.3.1.6 Configuring Alerts

Alerts can be managed via the Azure Portal and are used to send a notification to a particular Action Group when a pre-defined condition is triggered.

Rule Name	Conditions
Production Failed Requests	- A general alert that it will send an email once a new failure is detected via App Insights.
SFTP Process Failed	- Whenever the count StartFileDownloadProcess Failures is greater than 0; - Whenever the count StartFileDownloadProcess Failures is greater than 0
Inbound Transformation Process Failed	Whenever the StartInboundIntegrationProcess Azure Failures is greater than 0.
Production Exceptions	- A general alert that it will send an email once a new exception is detected via App Insights.

Name ↑↓	Condition	Severity ↑↓
☐ Failure Anomalies - appi-aif-env01	Failure Anomalies detected	3 - Informational
☐ Inbound Transformation Process Failed	StartInboundIntegrationProcess Failures > 0	1 - Error
☐ Production Exceptions	exceptions/count > 0	1 - Error
☐ Production Failed Request	requests/failed > 0	2 - Warning
☐ SFTP Process Failed	StartFileDownloadProcess Failures > 0 and 1 more con...	1 - Error

13 Security

13.1 VNET

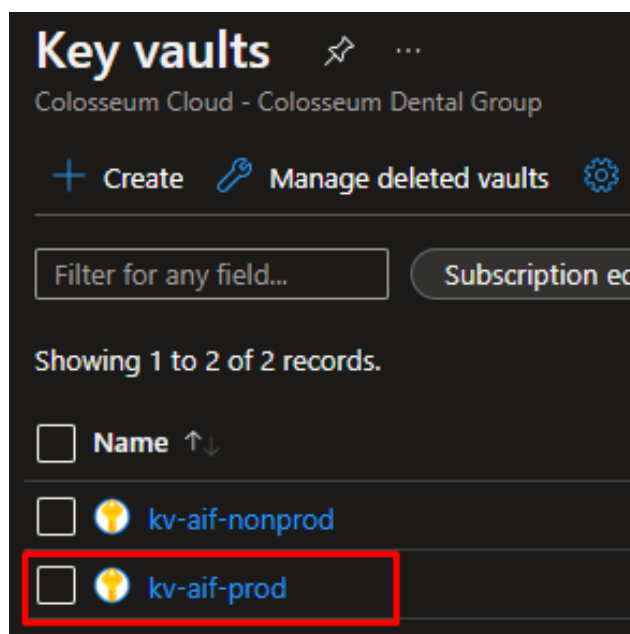
The VNET (Virtual Network) has Azure Firewall enabled. The firewall has Nat rules in place that will forward the traffic from the function apps to the traffic manager. The traffic manager will then forward all traffic to the CDG network using a single IP address. CDG network accepts only traffic from the traffic manager public ip.

13.2 Azure Key Vault

Key vault is used to store sensitive secrets such as passwords and access keys. Secrets in key vault are then directly referenced in Azure function or logic app configuration sections.

There are two key vault resources for CDG:

- **kv-aif-nonprod** is used for all non-production keys
- **kv-aif-prod** is used for production keys.



13.2.1 Key Vault Secrets

The following is a description of secrets that exist in production key vault. Similar secrets are found in the non-prod key vault as well.

Secret Name	Purpose
cdg-d365fo-aadClientId-integration-Production	The Procurement API password (used by Jaggaer)
cdg-api-jaggaer-username	The CDG Jaggaer Username API (Basic Authentication)
cdg-d365fo-aadClientId-integration-Production	The D365FO AAD App Registration (service principal) - client id
cdg-d365fo-aadClientSecret-integration-Production	The D365FO AAD App Registration (service principal) - client secret
cdg-d365fo-aadTenantId-integration-production	The D365FO AAD App Registration (service principal) - tenant id
cdg-infra-aadClientId-Terraform-Production	The AAD app registration (client ID) used to authenticate with the CDG Azure tenant from Terraform.
cdg-Infra-aadClientSecret-Terraform-Production	The AAD app registration (client secret) used to authenticate with the CDG Azure tenant from Terraform.
cdg-Infra-aadTenantId-Terraform-Production	The AAD app registration (tenant id) used to authenticate with the CDG Azure tenant from Terraform.
cdg-la-aif-env01-configuration-APPINSIGHTS-INSTRUMENTATIONKEY	The env01 Application Insights Instrumentation Key.

cdg-la-aif-env01-configuration-APPLICATIONINSIGHTS-CONNECTION-STRING	The env01 Application Insights connection string.
cdg-la-aif-env01-configuration-functionAppKey-masterkey	The env01 Azure Function master Function Key.
cdg-la-aif-env01-configuration-functionAppKey-outbound	The env01 Azure Function Outbound Function Key.
cdg-sftp-coupa-password	The Coupa SFTP password
cdg-sftp-coupa-ssh-private-key	The Coupa SFTP SSH private key
cdg-sftp-coupa-ssh-private-key-fingerprint	The Coupa SFTP SSH private key fingerprint. Currently Unused
cdg-sftp-coupa-ssh-private-key-passphrase	The Coupa SFTP SSH private key passphrase
cdg-sftp-henryschein-password	The HenrySchein SFTP Password.
cdg-sftp-henryschein-ssh-private-key	The HenrySchein SFTP SSH Private Key
cdg-sftp-mercur-password	The Mercur SFTP Password
cdg-sftp-opuscapita-password	The OpusCapita SFTP Password
cdg-sftp-pms-password	The PMS SFTP Password
cdg-sftp-tagetik-password	The Tagetik SFTP Password

cdg-st-accessKey-env01	The storage account access key.
cdg-st-accessKey-env01-func	The storage account access key
cdg-st-accessKey-env01-funcconnstr	The connection string of the storage account hosting the Azure function.
cdg-st-accessKey-env01-logic	The storage account key of the storage account hosting the Logic App.
cdg-st-accessKey-staifprodterraform-key1	The storage account access key 1 that its being used by Terraform to handle states.
cdg-st-connectionString-env01	The env01 storage account connection string.
cdg-st-logicappconnectionString-env01	The storage account connection string hosting the Logic Apps.
jaggaer-api-endpoint-clientId	The Jaggaer REST API Client ID
jaggaer-api-endpoint-clientSecret	The Jaggaer REST API Client Secret

13.3 Application Registrations

13.3.1 List of Application Registrations

The purpose of an App Registration in an Azure Active Directory is to authenticate several services to the CDG Azure Active Directory.

Please refer to the [Azure Key Vault section](#) to view the list of available secrets. The Key Vault secrets which hold the Application Registration secrets contain the same naming convention of the Application Registration to make it easier for the BAU to identify it.

The below list contains only Application Registrations which are being used by the Azure middleware's.

App Registration	Purpose
CDG Azure Terraform Integration – NonProduction	To authenticate with the CDG Azure Non-Production subscription from the Terraform script.
CDG Azure Terraform Integration – Production	To authenticate with the CDG Azure Production subscription from the Terraform script.
CDG AzureFunction Integration – ENV03 – NonProduction	To authenticate with the CDG Azure Non-Production Subscription from the ENV03 Azure Function.
CDG AzureFunction Integration – ENV04 – NonProduction	To authenticate with the CDG Azure Non-Production Subscription from the ENV04 Azure Function.
CDG AzureFunction Integration – ENV05 – NonProduction	To authenticate with the CDG Azure Non-Production Subscription from the ENV05 Azure Function.
CDG AzureFunction Integration – ENV06 – NonProduction	To authenticate with the CDG Azure Non-Production Subscription from the ENV06 Azure Function.
CDG AzureFunction Integration – ENV01 –Production	To authenticate with the CDG Azure Production Subscription from the ENV01 Azure Function.
CDG D365FO Integration – NonProduction	To authenticate with D365FO from the Non-Production Azure Logic Apps. All the non-production middleware environments are authenticating with D365FO using this App Registration.
CDG D365FO Integration – Production	To authenticate with D365FO from the Production Azure Logic App.

13.4 SSL Certificate

13.4.1 Configuration of SSL Certificate

CDG uses a customer domain and therefore an SSL certificate is needed. The SSL Certificate is located in the production key vault. The certificate provider is DigiCert. The certificate expires towards the beginning of April.

kv-aif-prod | Certificates

Key vault

Search

Generate/Import Refresh Restore Backup Manage deleted certificates Certificates

Name	Thumbprint	Status
Completed		
wildcardstar-colosseumcloud-com	7A7AB4A3ECFEB1E66A3F737EFC3B4DC...	✓ Enabled
In progress, failed or cancelled		
There are no certificates available.		

To renew the certificate:

- 1) Click on the certificate (*wildcardstar-colosseumcloud-com*).
- 2) Click on Certificate Operation

Home > kv-aif-prod | Certificates >

wildcardstar-colosseumcloud-com

Versions

New Version Refresh Delete Download Backup Issuance Policy Certificate Operation

Version	Thumbprint	Status
CURRENT VERSION		
a367fc9332884455977fc97a2b2727f0	7A7AB4A3ECFEB1E66A3F737EFC3B4DCEBC5E49FC	✓ Enabled
OLDER VERSIONS		
627be6a48d8a441d9c7bc130ead180c5	C0DB400C7DA38CEE08CBE81CC06C868D2E4E3D16	✓ Enabled

- 3) Click on Download CSR

wildcardstar-colosseumcloud-com

Certificate Operation

Refresh Download CSR Merge Signed Request Request Cancellation Delete

Status ✓ Completed

Request ID 40990c3eb4de41c18134502e452e9e13

CA Name DigiCertCA

- 4) Once the CSR is downloaded, it must be renewed at DigitCert.
- 5) Once the certificate is renewed, and the new version is available, open again the certificate and click on New Version.

Home > kv-aif-prod | Certificates >

wildcardstar-colosseumcloud-com

Versions

New Version Refresh Delete Download Backup Issuance Policy Certificate Operation

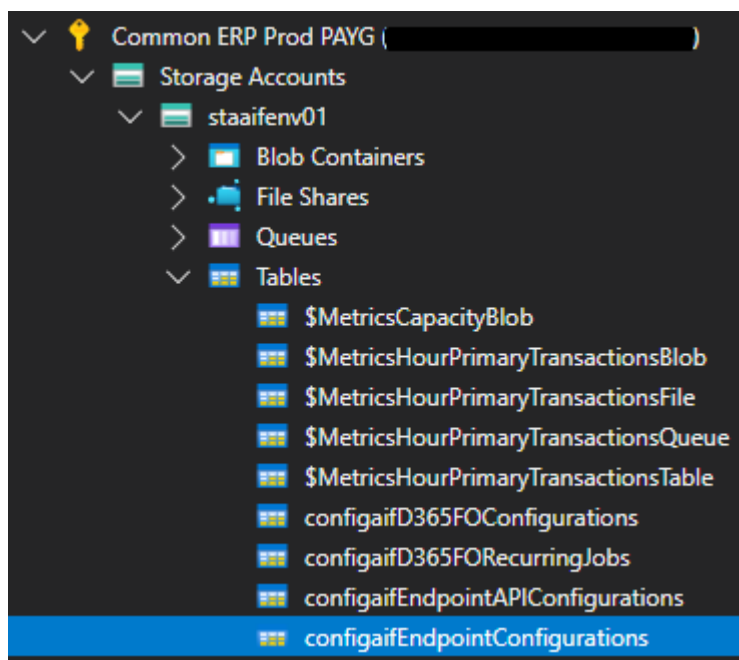
Version	Thumbprint	Status
CURRENT VERSION		
a367fc9332884455977fc97a2b2727f0	7A7AB4A3ECFEB1E66A3F737EFC3B4DCEBC5E49FC	✓ Enabled
OLDER VERSIONS		
627be6a48d8a441d9c7bc130ead180c5	C0DB400C7DA38CEE08CBE81CC06C868D2E4E3D16	✓ Enabled

- 6) Import the certificate. Once imported the new version of the certificate will show up in the Current Version section.

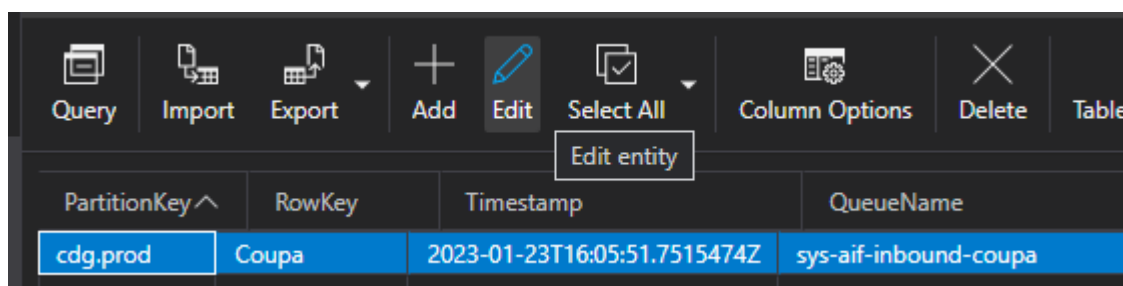
14 FAQs

14.1 How to change the Third Party credentials?

Go to the **configaifEndpointConfigurations** table under the **staaifenv01** storage account from the Azure Storage Explorer or the Azure Portal.



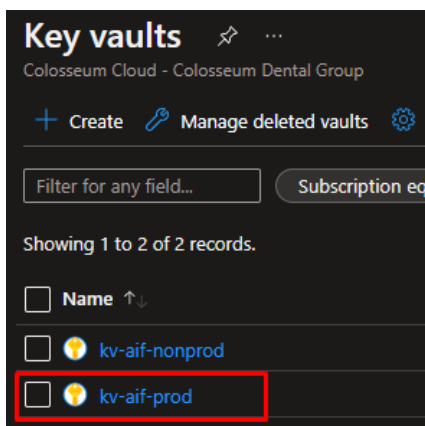
The *RowKey* represents the 3rd party. Edit the record you want to update.



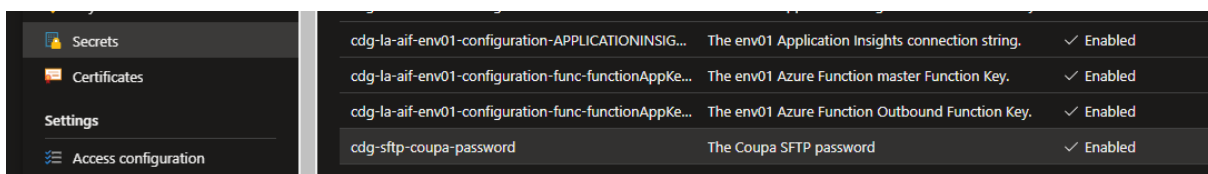
You can update the following fields here:

- SFTPServerURL
- SFTPUser

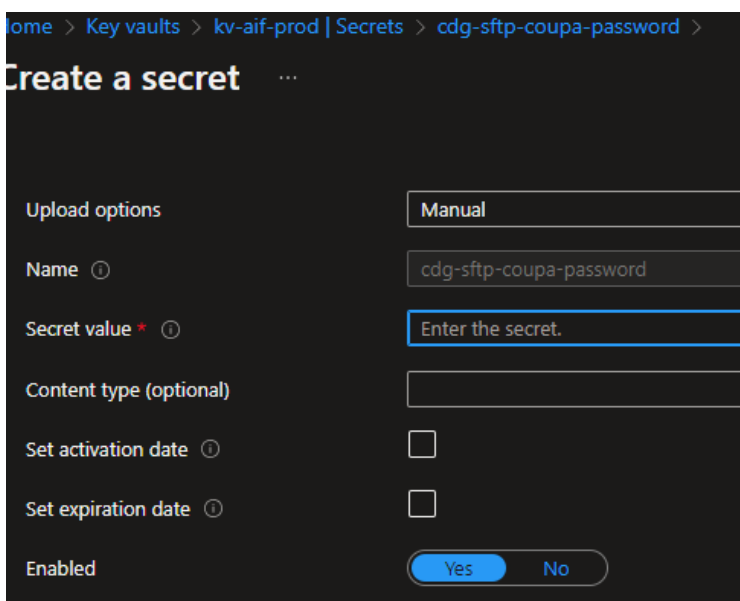
Passwords or SSH Keys are not stored in this table but in KeyVault. From the Azure Portal, open the **kv-aif-prod** keyvault.



Open the secrets section and click on the secret you want to update. They all follow the same naming convention, e.g., **cdg-sftp-coupa-password**.



Click on **New Version**, enter the new value and click **Create**. The middleware will always take the latest version.



14.2 How to run an integration manually?

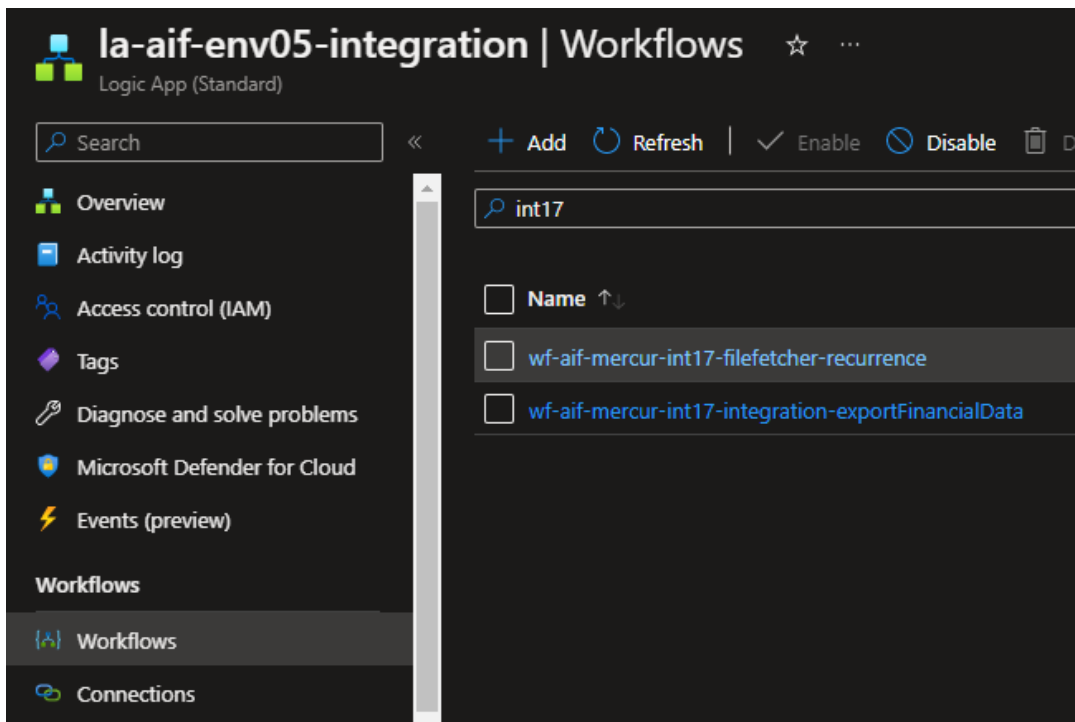
For an integration to work, data must exist at the source. The following example is an example of triggering an INT17 (Mercur - outbound) integration on ENV05.

1) Make sure that data is ready to be picked up in the D365FO instance.

2) Go to the relevant logic app resource (la-aif-env05-integration).

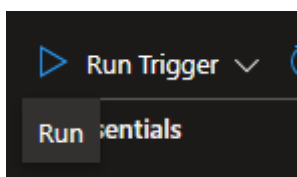
☐ Name ↑↓	Status ↑↓	Plan ↑↓	Resource group ↑↓
☐ la-aif-env01-integration	Running	Standard	rg-aif-prod-env01
☐ la-aif-env03-integration	Running	Standard	rg-aif-nonprod-env03
☐ la-aif-env04-integration	Running	Standard	rg-aif-nonprod-env04
☐ la-aif-env05-integration	Running	Standard	rg-aif-nonprod-env05

3) Go to the Workflows tab and search for int17.



4) Open the wf-aif-mercur-int17-filefetcher-recurrence.

5) Click on the drop down next to Run Trigger and click on Run.



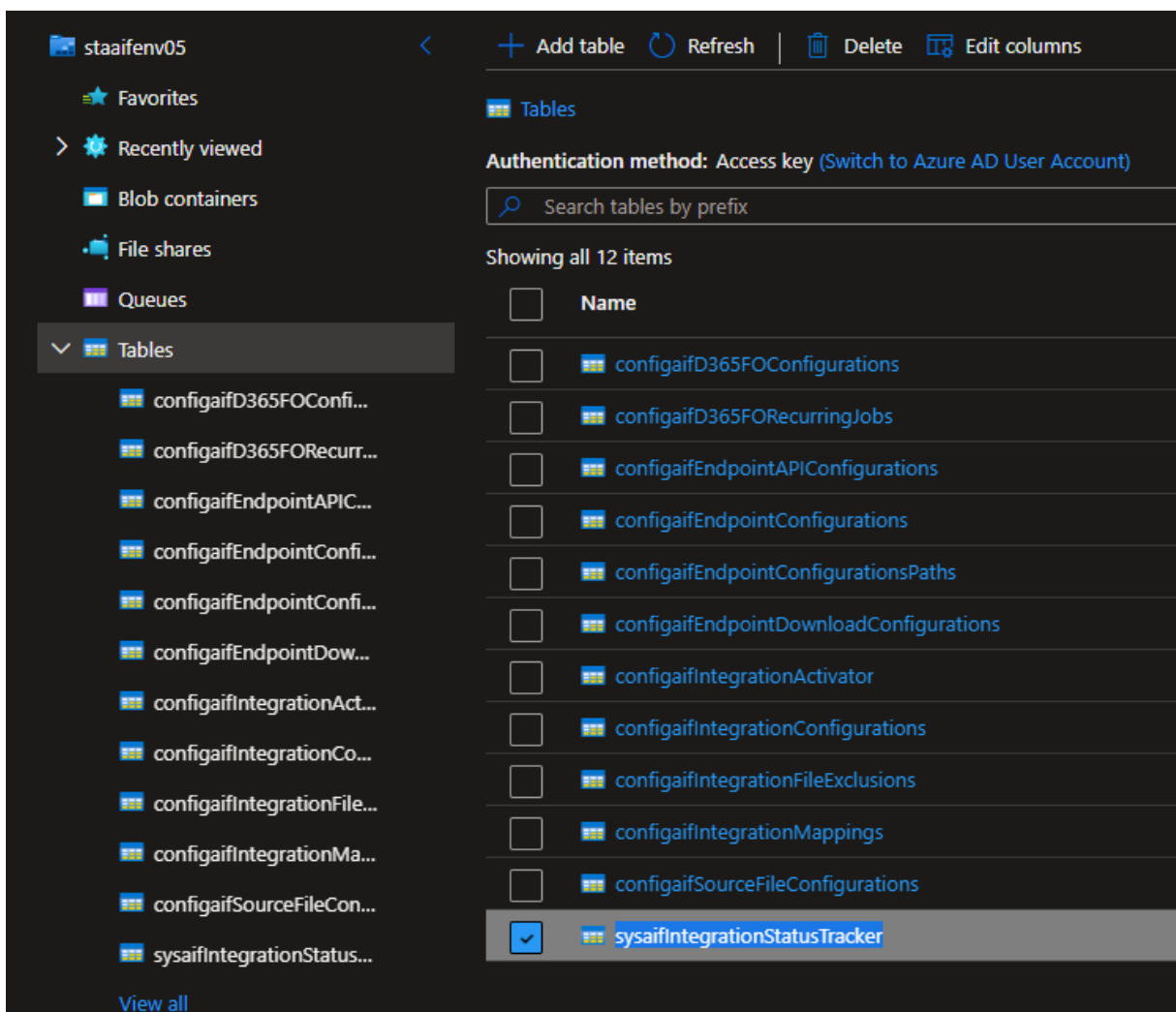
14.3 How to monitor an integration?

The following example is a continuation of running an integration manually and as an example is in context of an INT17 – Mercur outbound integration on env05.

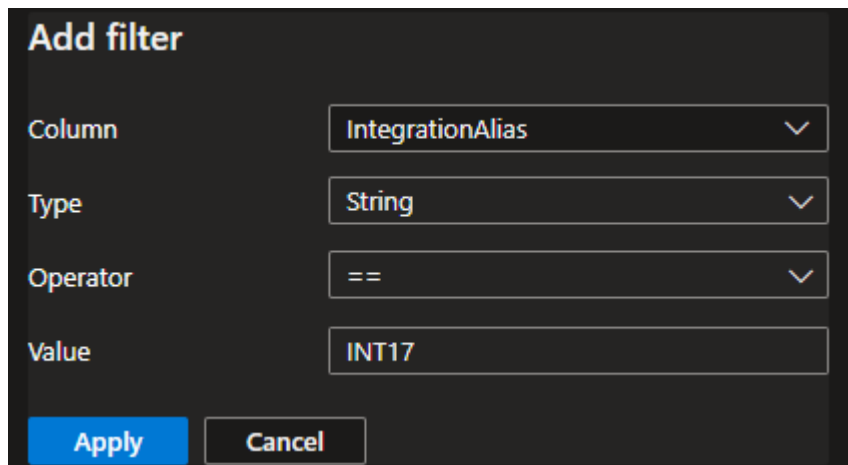
- 1) Go to the storage account of the specific environment (staaifenv05)



- 2) Go to Storage browser, then Tables. Click on the *sysaifIntegrationStatusTracker* table.



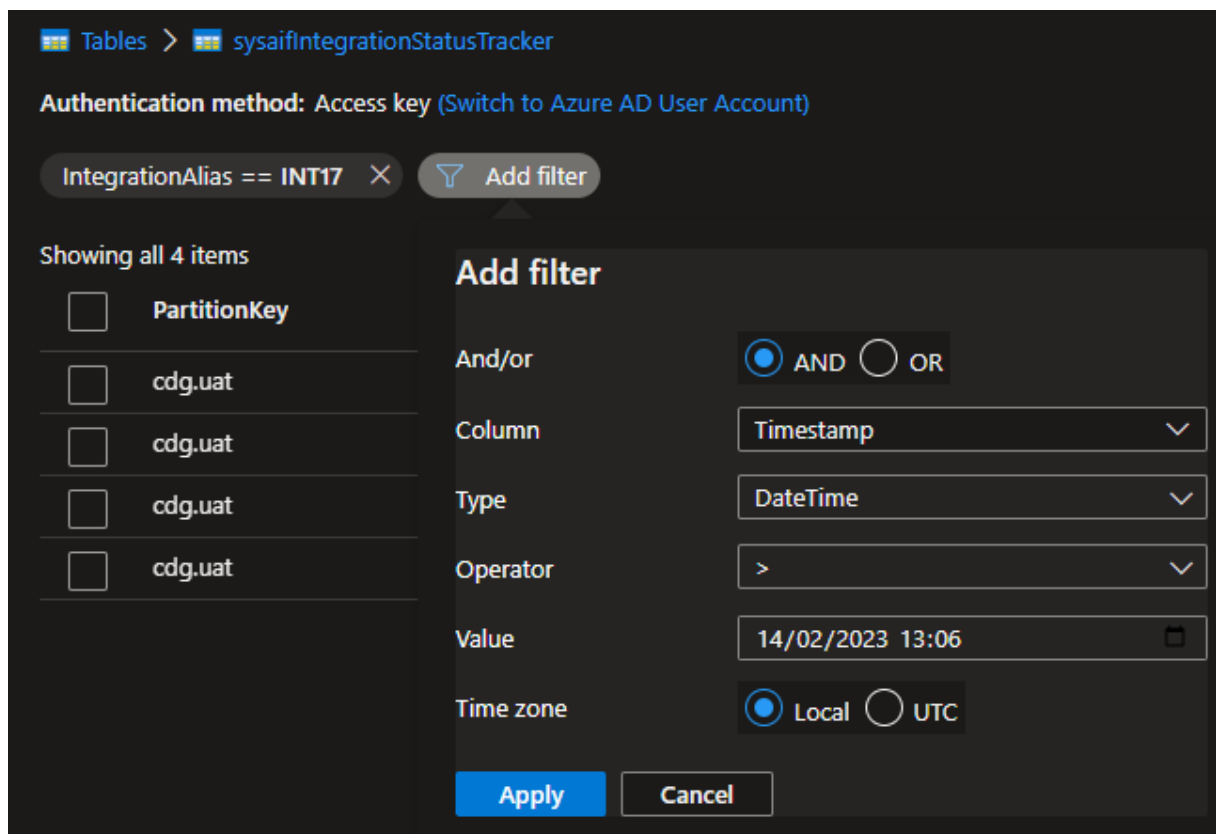
3) Click on Add filter and add a filter for the Integration Alias and for the timestamp to show the last few minutes.



The 'Add filter' dialog box is shown with the following fields:

Column	IntegrationAlias
Type	String
Operator	==
Value	INT17

Buttons: Apply, Cancel



The main application interface shows the 'sysaifIntegrationStatusTracker' table. The authentication method is 'Access key'. A filter 'IntegrationAlias == INT17' is applied. The table shows 4 items, all with 'cdg.uat' as the PartitionKey. The 'Add filter' dialog box is open, showing the following fields:

And/or	☒ AND ☐ OR
Column	Timestamp
Type	DateTime
Operator	>
Value	14/02/2023 13:06
Time zone	☒ Local ☐ UTC

Buttons: Apply, Cancel

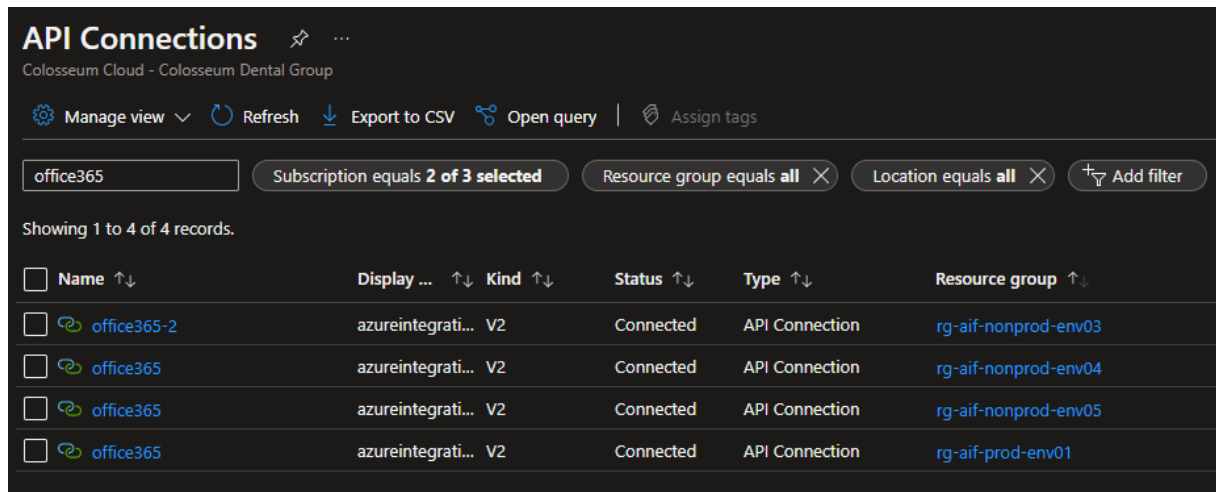
4) A record should exist representing the latest file picked up. Notice the AIFIntegrationStatus which can have the value of Pending / InProgress / Transformed / Successful / Failed. Wait until the status is either Successful or Failed. This can take a few minutes.

Please refer to [Integration Process Flow](#) for further information about the integration process.

14.4 Emails are not being sent anymore. What should I do?

At a point, the emails may stop being sent and need to be authorised. You might also see errors in workflows that contain actions which send alert emails such as **wf-aif-core-handleError** and **wf-aif-core-d365fo-monitoring**.

To fix this, go to **API Connections** in the Azure portal and search for **office365**.

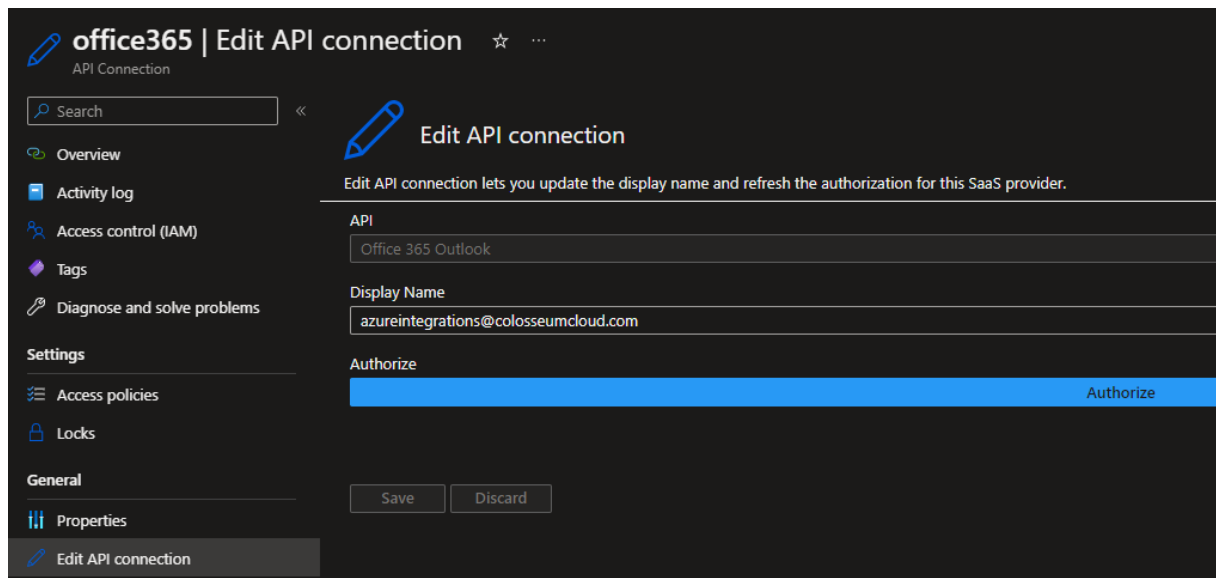


The screenshot shows the 'API Connections' page in the Azure portal for the 'Colosseum Cloud - Colosseum Dental Group' subscription. It displays a table with 4 records for 'office365' API connections. All connections are in a 'Connected' status and are of type 'API Connection'.

Name	Display Name	Kind	Status	Type	Resource group
office365-2	azureintegrati...	V2	Connected	API Connection	rg-aif-nonprod-env03
office365	azureintegrati...	V2	Connected	API Connection	rg-aif-nonprod-env04
office365	azureintegrati...	V2	Connected	API Connection	rg-aif-nonprod-env05
office365	azureintegrati...	V2	Connected	API Connection	rg-aif-prod-env01

Open the one which is relevant for your environment (see Resource group) and go to **Edit API Connection**.

Click on **Authorize** and use azureintegrations@colosseumcloud.com to authorize.



The screenshot shows the 'Edit API connection' page for 'office365'. The 'API' field is set to 'Office 365 Outlook'. The 'Display Name' field is set to 'azureintegrations@colosseumcloud.com'. The 'Authorize' button is highlighted in blue.

office365 | Edit API connection

API: Office 365 Outlook

Display Name: azureintegrations@colosseumcloud.com

Authorize: [Authorize]

Save Discard

! Please note that if this is authorized using a different email address, the emails may still not be sent.

14.5 How to Approve an Azure DevOps Release?

To approve an Azure DevOps release, the release manager will have to follow the below steps:

1. Open the Azure DevOps release.
2. Locate the latest release that needs to be approved.
 - a. Note: Normally the release approvers will receive an email as soon as a new release is issued and is awaiting approval. If you haven't received an email or you are not configured as a release approval, you will not be allowed to approve a release.
3. Click on the Release that requires approval.

Deploy CDG AIF Middleware Transformations Function

Releases Deployments Analytics

Pending approval on **ENV01 PRODUCTION - Transformation** stage.

Releases	Created	Stages
Release-93 20230315... master	15/03/2023, 11:08:07	ENV03-Tr... ENV04-Tr... ENV05-Tr... ENV01 PR...

4. As a best practice, the approver can enter a comment prior approving the release. To approve the release, the approver must click on the **Approve** button. Similarly, to reject a release, the approver will have to click on the **Reject** button.

Deploy CDG AIF Middleware Transformations Function

Releases Deployments Analytics

Pending approval on **ENV01 PRODUCTION - Transformation** stage.

Release-93 > Pending approval on ENV01 PRODUCTION - Transformation stage.

Pre-deployment approval pending [View logs](#)

Approval pending for 5 days
Waiting for any one of the approvers to approve. Timeout in 25d

Approve **Reject**

Kevin Agius Pending for 5 days [Reassign](#)

Comment

☐ Defer deployment for later

Mahesh Ambati Pending for 5 days [Override](#)

Sandeep Basavanyappa Pending for 5 days [Override](#)

Benjamin Axiq Pending for 5 days [Override](#)

Jesmond.Darmanin Pending for 5 days [Override](#)

Other remarks to consider:

- Each release has a lifetime. By default a release has a timeout of 30 days prior it is expired. If the release is expired, the release manager will have to create a new release.
- The approver can re-assign the approval process to another individual (the other individual has to have access to approve).

14.6 How to restart a Logic App?

Restarting a Logic App Standard is generally not necessary or recommended unless there is a specific need or issue that requires it.

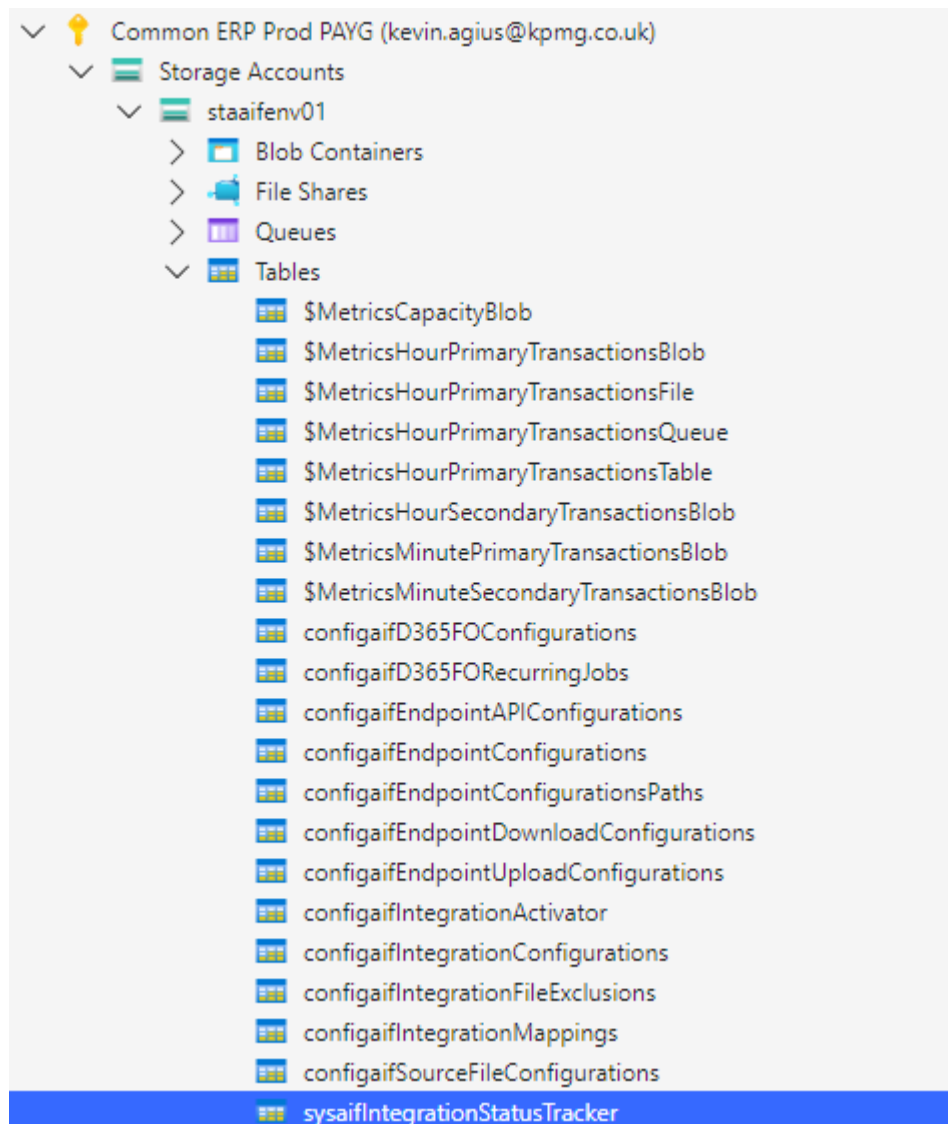
It is important to rely on the built-in capabilities and monitoring tools provided by Azure to handle automatic restarts, handle errors, and troubleshoot any issues that arise.

It is recommended that prior to restarting the Logic App; the BAU engineer will verify that there are no integrations running.

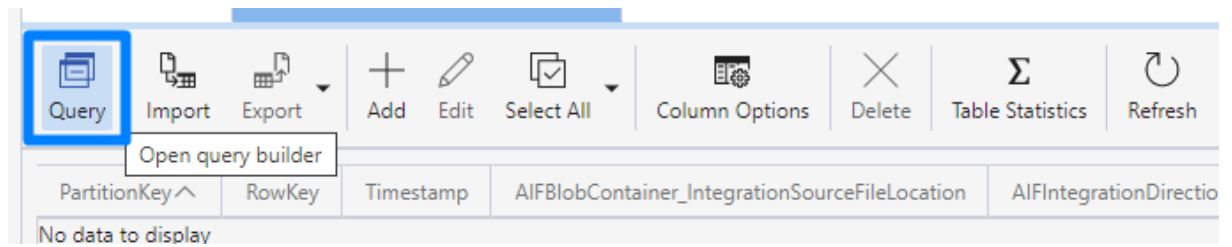
Below you will find the steps required to check whether there are any integrations running and the steps needed to restart the workflow.

14.6.1 Verify that there are no integrations running

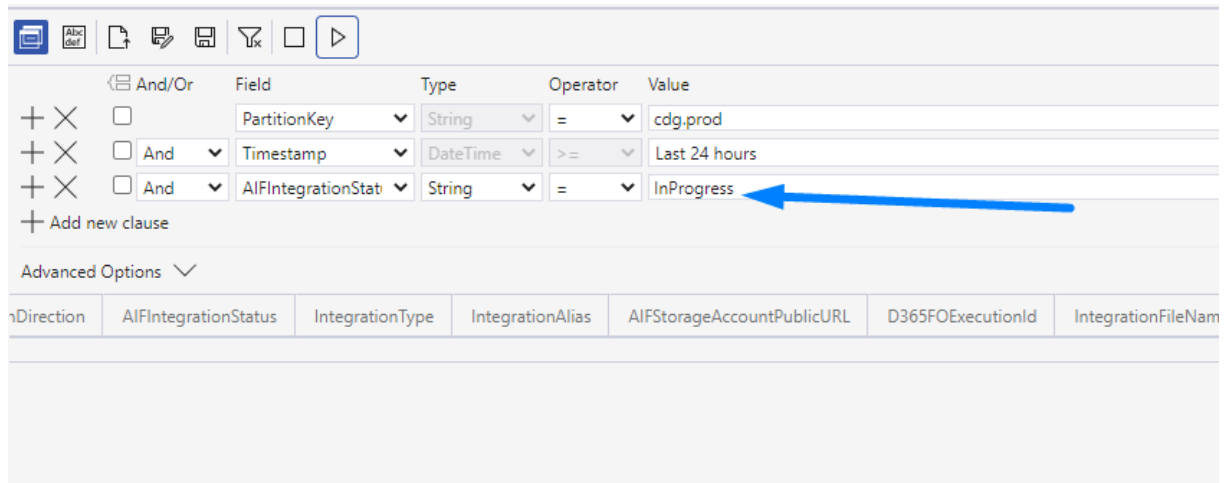
- a) Open the Storage Account Explorer
- b) Open the Storage Account > **Tables** > Open the '*sysaifIntegrationStatusTracker*' table.



c) Open the Query



d) Add the filter for the *AIFIntegrationStatus* field and type 'InProgress' in the Value



Note: Integrations with their status 'InProgress' are integrations which are currently been executed.

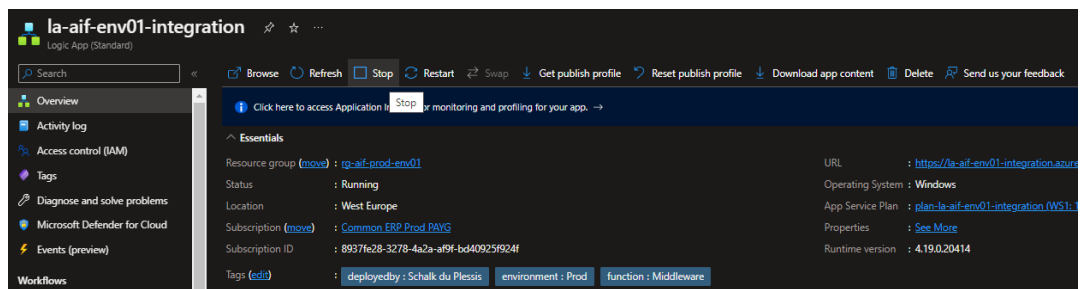
If no integrations are displayed in the search results, as shown in the previous screenshot, it indicates that there are currently no running integrations. In such cases, it is considered safe to proceed with the next step of [restarting the Logic App](#).

However, if you come across any records with an 'InProgress' status, it is recommended to wait for a few minutes and attempt the search again. If, after this waiting period, the record remains in the 'InProgress' status and the workflows appear to be stuck, make a note of the integration details, and proceed to the next step of [restarting the Logic App](#).

After restarting the Logic App, locate the integration runs that have failed and adjust their status and paths to put them in a state where they can be re-triggered. It is important to note that each situation may be unique, so if you encounter any difficulties during the process, it is recommended to contact KPMG for assistance.

14.6.2 Restarting an Azure Logic App

- i) Open the Azure portal and click on the **Logic Apps** service to locate the required Logic App. (i.e., The Production logic app name is 'la-aif-env01-integration')
- ii) To initiate a soft restart, you can click on the '**Restart**' option. However, in situations where the logic app workflows are not triggering for some reason, it is advisable to perform a hard restart. For a hard restart, you need to click on '**Stop**', wait for the service to completely stop, and then click on '**Start**'. Please allow approximately 2-3 minutes for the Logic App to fully start.



14.6.3 Verify that the integrations resumed their execution

There are several ways of how the BAU engineer can verify that the integration runs have resumed their execution; but one of the ways is the below.

- (i) Open the Storage Account Explorer
- (ii) Open the Storage Account > Tables > Open the '**sysaifIntegrationStatusTracker**' table.
- (iii) Set the *Timestamp* for '**Last 24 hours**'
- (iv) Make sure that the records are being updated from 'InProgress' to **ExecutingInD365FO / Successful / Failed** and are not stuck in 'InProgress' status for a long time.

And

Or

And

Or

And

Or

And

Or

Field

PartitionKey

Timestamp

Type

String

DateTime

Operator

=

>=

Value

cdg.prod

Last 24 hours

+

×

□

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

And

+

×

□

Contact us

Kevin Agius

Azure Engineering Technical Lead

E kevin.agius@kpmg.co.uk

Benjamin Axiaq

Azure Engineering Technical Architect

E benjamin.axiaq@kpmg.co.uk

Jesmond Darmanin

Azure Infrastructure Technical Architect

E jesmond.darmanin@kpmg.co.uk

www.kpmg.com

© 2023 ,

The information contained herein is of a general nature and is not intended to address the circumstances of any particular individual or entity. Although we endeavour to provide accurate and timely information, there can be no guarantee that such information is accurate as of the date it is received or that it will continue to be accurate in the future. No one should act on such information without appropriate professional advice after a thorough examination of the particular situation.

The KPMG name and logo are registered trademarks or trademarks of KPMG International Cooperative.

